

PRINT-FLOW-360 · RESEARCH LIBRARY

English

13 research documents · each with plain-language key takeaways

Compiled 2026-06-26

Table of Contents

1. Developer English — Grammar Course
2. 01 — Articles: a, an, the (and no article)
3. 02 — Countable vs uncountable: much / many / few / less / fewer
4. 03 — Subject–verb agreement
5. 04 — Verb tenses & keeping them consistent
6. 05 — Plurals, apostrophes & possessives (its vs it's)
7. 06 — Prepositions (in, on, at, to, for, with)
8. 07 — Punctuation & capitalization basics
9. 08 — Sentence structure: run-ons, fragments & comma splices
10. 09 — Commonly confused words
11. 10 — Professional dev writing: commits, PRs, Slack, comments
12. 11 — Spelling & typos developers get wrong
13. 12 — Master mixed review

Developer English — Grammar Course

Source: [readme/research/english/00-index.md](#)

IN SHORT — KEY TAKEAWAYS

This is a short English writing course made for you. It focuses on the exact mistakes you make in your everyday work writing — things like spelling errors, missing punctuation, and texting habits that slip into professional messages. Fixing these small, repeated slips will make your commits, messages, and pull requests much easier for others to read and trust.

The main parts explained simply:

- **Your top 7 mistakes** — The course starts by naming the exact errors you make most often, with a real before-and-after example. Knowing your specific weak spots means you study what you actually need, not things you already get right.
- **Articles (a, an, the)** — These tiny words trip up almost every non-native speaker. This lesson tells you when to use each one, and when to use none at all.
- **Much vs many / countable words** — “So much mistake” should be “so many mistakes.” This lesson explains why — some words can be counted (mistakes, files, orders) and some cannot (water, work), and that changes which word you use.
- **Subject–verb agreement** — Making the verb match who or what is doing the action. “He run” is wrong; “He runs” is right. A quick rule set that fixes this for good.
- **Verb tenses** — Past, present, and future — and how to stop switching between them in the same sentence without realising it.
- **Plurals and possessives** — When to add s, when to add 's, and the classic trap: “its” (belonging to it) vs “it’s” (it is).
- **Prepositions (in, on, at, for, with)** — The small linking words that native speakers use without thinking. This lesson gives you the most common patterns.
- **Punctuation and capitalization** — Full stops, question marks, commas — and the rule that “I” is always a capital letter, no exceptions.
- **Sentence structure** — How to break one long run-on sentence into clear, separate ideas so readers do not lose the thread.
- **Commonly confused words** — their / there / they’re, your / you’re, then / than. One lesson to sort them out once and for all.
- **Professional writing for developers** — How to write commits, pull requests, Slack messages, and code comments without texting habits (no “u”, “r”, “gonna”). Work writing has its own standard.
- **Spelling and typos** — The words engineers misspell most, plus how to let spell-check tools catch the rest automatically.
- **Master review (Lesson 12)** — One big practice drill that covers everything from all eleven lessons together. Use it to check how much has stuck.

What to do with this: Study one lesson at a time, spread over four weeks — about 20 to 30 minutes a day, a few days a week. Always try the practice drills yourself before checking the answers. A wrong answer you then correct teaches you far more than one you just read.

A practical grammar course built around the real writing mistakes you make, so your commits, PRs, and messages read clearly and professionally.

Why this course exists

This course is not a generic grammar textbook. It's built around **your actual recurring mistakes** — the ones that show up in your day-to-day writing. Instead of teaching you rules you'll never use, we focus on the handful of patterns that trip you up again and again:

- Spelling typos (`reaseach` , `grammer`)
- Lowercase "i" instead of capital I
- Run-on sentences with no punctuation
- Texting style (`u` instead of "you") in work writing
- `much` vs `many` , and matching plurals
- Missing question marks and full stops

Here's the real before/after that started this whole thing:

Before: "can u do reaseach on grammer for me and teach me i already know but as of now i am making so much mistake"

After: "Can you do research on grammar for me and teach me? I already know it, but right now I'm making so many mistakes."

Look at everything that got fixed in that one sentence:

What changed	Before	After	Why
Spelling	<code>reaseach</code> , <code>grammer</code>	research, grammar	Common typos — spell-check catches these
Texting style	<code>u</code>	you	"u" is fine in texts, not in work writing
Capital I	<code>i</code>	I	The word "I" is always capitalized
Punctuation / run-on	one long line	split into clear sentences with <code>?</code> and <code>.</code>	A question needs a <code>?</code> ; separate ideas need separate sentences
much → many + plural	<code>so much</code> <code>mistake</code>	so many mistakes	"mistakes" is countable, so use many and make it plural

Every lesson below targets one of these patterns directly.

Your top 7 mistakes

Mistake	Example you wrote	The fix	Lesson
Texting style in work writing	can u do...	"Can you do..."	10-professional-dev-writing.md
Lowercase "i"	i already know	"I already know"	07-punctuation-and-capitalization.md
Spelling typos	reaseach, grammer	research, grammar	11-spelling-typos.md
Run-on sentences	...teach me i already know but...	Break into separate sentences	08-sentence-structure-run-ons.md
Missing end punctuation	...teach me (it's a question)	"...teach me?"	07-punctuation-and-capitalization.md
much vs many	so much mistake	"so many mistakes"	02-countable-uncountable-much-many.md
Plural agreement	mistake (should be plural)	"mistakes"	05-plurals-and-possessives.md

The lessons

1. **Articles: a, an, the (and no article)** — when to use *a*, *an*, *the*, or nothing at all.
2. **Countable vs uncountable: much / many / few / less / fewer** — fixes the "so much mistake" pattern for good.
3. **Subject-verb agreement** — making the verb match the subject (he *runs*, they *run*).
4. **Verb tenses & keeping them consistent** — past, present, future, and not switching mid-sentence.
5. **Plurals, apostrophes & possessives (its vs it's)** — when to add *s*, *'s*, or just *'*.
6. **Prepositions (in, on, at, to, for, with)** — the small words that are easy to get wrong.
7. **Punctuation & capitalization basics** — full stops, question marks, commas, and always capitalizing **I**.
8. **Sentence structure: run-ons, fragments & comma splices** — one idea per sentence, joined the right way.
9. **Commonly confused words** — their/there/they're, your/you're, then/than, and more.
10. **Professional dev writing: commits, PRs, Slack, comments** — clear, no-texting-style writing for work.
11. **Spelling & typos developers get wrong** — the words that trip up engineers (and how spell-check helps).
12. **Master mixed review** — one big drill covering everything.

How to study

Don't rush all twelve lessons in one sitting. Spread them over **four weeks** — about 20-30 minutes a day, a few days a week. Here's a plan:

- **Week 1 — the fundamentals.** Lessons 01, 02, and 03. These cover articles, the much/many fix, and subject–verb agreement.
- **Week 2 — verbs and words.** Lessons 04, 05, and 06. Tenses, plurals/possessives, and prepositions.
- **Week 3 — structure and clarity.** Lessons 07, 08, and 09. Punctuation, run-ons, and confusing word pairs.
- **Week 4 — make it professional, then review.** Lessons 10, 11, and the master mixed review.

How to use the drills: Each lesson ends with practice drills. Do the whole drill *first* — write your answers down or say them out loud — and *only then* check the answer key. Don't peek early; the value is in trying, even when you get it wrong. A wrong answer you then correct sticks far better than one you just read.

Spaced-repetition tip: Memory fades fast. A few days after finishing a lesson, come back and **redo the same drills**. If you still get them right, great — that knowledge is locked in. If you slip, that's exactly the lesson to review again. Redoing drills a few days later is the single highest-value habit in this course.

Daily habit

Small habits beat big study sessions. Pick these up and your writing improves automatically:

- **Always capitalize “I”** — the word “I” is *never* lowercase.
- **End every sentence with a full stop** (or a `?` for questions). No naked line endings.
- **Read your message once before you send it** — one quick pass catches most mistakes.
- **Turn on spell-check** in your editor, browser, and chat apps. Let the computer catch typos like `grammer`.
- **No texting style in work writing** — write “you” not `u`, “are” not `r`. Save the shortcuts for texts with friends.

Progress checklist

- [] 01 — Articles: a, an, the (and no article)
- [] 02 — Countable vs uncountable: much / many / few / less / fewer
- [] 03 — Subject–verb agreement
- [] 04 — Verb tenses & keeping them consistent
- [] 05 — Plurals, apostrophes & possessives (its vs it's)
- [] 06 — Prepositions (in, on, at, to, for, with)
- [] 07 — Punctuation & capitalization basics
- [] 08 — Sentence structure: run-ons, fragments & comma splices
- [] 09 — Commonly confused words
- [] 10 — Professional dev writing: commits, PRs, Slack, comments
- [] 11 — Spelling & typos developers get wrong
- [] 12 — Master mixed review

You already know more grammar than you think — the goal here is just to stop the small, repeated slips. One lesson at a time. You've got this.

01 — Articles: a, an, the (and no article)

Source: [readme/research/english/01-articles-a-an-the.md](#)

IN SHORT — KEY TAKEAWAYS

This document teaches you when to write “a”, “an”, “the”, or nothing before a word in English. Getting this right makes your messages, pull requests, and comments sound natural and professional — not like a log file.

The main parts explained simply:

- **The three choices (a / an / the / nothing)** — “a” or “an” means one of many things, or the first time you mention something. “the” means a specific thing both you and the reader already know. No article at all is for general ideas or uncountable things like “data”, “code”, or “feedback”.
- **Sound, not spelling** — You pick “a” or “an” based on how the word *sounds*, not how it is spelled. “an SQL query” (because SQL sounds like “ess”), “a URL” (because URL sounds like “you-arr-ell”), “an honest mistake” (because the “h” is silent — it sounds like “onnest”).
- **Why developers drop articles** — When writing code, commit messages, or quick messages, people skip small words to go faster. This is a habit that carries into real sentences and makes them sound robotic or incomplete.
- **When “the” is wrong** — Do not use “the” with uncountable words (“the data”, “the feedback”, “the code”) or with tool and product names (“the GitHub”, “the Slack”, “the Redis”). Use “the” only when you mean one specific known thing.
- **Commit messages are the exception** — In a commit message, dropping articles is accepted style (`fix null check in cart service` is fine). But in PR descriptions, Slack messages, and code comments, write full sentences with articles.
- **Common mistakes fixed** — The document shows a table of real examples, like “open pull request” → “open **a** pull request”, and “send the feedback in the slack” → “send **feedback** in **Slack**”, with a clear reason for each fix.
- **Practice drills** — Twelve sentences to fix or fill in, with a full answer key explaining every choice. Good for building the habit through repetition.

What to do with this: Use the quick rule — *specific and known* → *the*; *first mention or one of many* → *a/an*; *general or uncountable* → *nothing* — whenever you write a Slack message, PR description, or code comment. Keep the drill answers handy and check your writing against the “See it / fix it” table until the pattern feels automatic.

*What this fixes for you: the dropped “a/an/the” in your commits and PRs (“open pull request” → “open **a** pull request”), and the “the” you add where English wants nothing.*

The rule in 30 seconds

- **a / an** = one of many, or the first time you mention it. Use **an** before a *vowel sound*, **a** before a *consonant sound* — it's about how the word sounds, not how it's spelled.
- **the** = a specific thing both you and the reader already know about (the one we just mentioned, or the only one there is).
- **no article** = general plurals ("tests fail"), uncountable stuff ("data", "code", "feedback"), and most names (Redis, GitHub, `main`).
- Quick flow: **specific & known** → **the**. **First mention / one of many** → **a/an**. **General plural or uncountable** → **no article**.

Why this trips you up

In code and chat you drop short words to go fast — "open pull request", "merge to main branch", "fix bug in service" — so the articles fall off and the sentence reads like a log line, not English. The same habit makes you write "the data" or "the feedback" when English wants no article at all. Sound vs. letter is the other snag: it's "an SQL query" (sounds like *ess*) but "a URL" (sounds like *you-arr-ell*).

See it / fix it

✗ What you tend to write	✓ Correct	Why
open pull request for this fix	open a pull request for this fix	First mention, one of many PRs → a .
merge to main branch after review	merge to the main branch after review	There is only one main → specific & known → the .
i opened PR, can u review	I opened a PR. Can you review it ?	a PR (first mention), capital I , full word you , and it refers back.
fix bug in payment service	fix the bug in the payment service	A specific known bug and one known service → the both times.
this throws error when token is empty	this throws an error when the token is empty	“error” = first mention an ; sounds like <i>err</i> . The token is the specific one in context → the .
we need write unit test for this	we need to write a unit test for this	One of many tests → a (and “to write”).
restart server, then run migration	restart the server, then run the migration	Both are the specific known ones in this task → the .
i am making so much mistake with article	I am making so many mistakes with articles	“mistake” is countable → many + plural -s ; general plural “articles” → no article.
send the feedback in the slack	send feedback in Slack	“feedback” is uncountable → no article; “Slack” is a proper name → no article.
it returns a 401 unauthorized error	it returns a 401 Unauthorized error	“401” sounds like <i>four-oh-one</i> (consonant sound) → a . ✓ already fine — keep it.
this is honest mistake in reaseach	this is an honest mistake in research	“honest” — the <i>h</i> is silent, sounds like <i>onnest</i> → an ; spelling: research.

Patterns to remember

a vs an — listen, don’t spell:

Word	Sound starts with	Article
URL	<i>you</i> (consonant sound)	a URL
SQL query	<i>ess</i> (vowel sound)	an SQL query
HTTP request	<i>aitch</i> (vowel sound)	an HTTP request
hour	<i>our</i> (silent h)	an hour
user	<i>you</i> (consonant sound)	a user
API	<i>ay</i> (vowel sound)	an API
8-character token	<i>eight</i> (vowel sound)	an 8-character token

When NOT to use “the”:

- Uncountable things in general: ~~the~~ code, ~~the~~ data, ~~the~~ feedback, ~~the~~ documentation, ~~the~~ information.
- General plurals: “~~the~~ tests pass”, “we ship ~~the~~ features weekly”.
- Most proper names: ~~the~~ GitHub, ~~the~~ Redis, ~~the~~ `main`, ~~the~~ Docker.
- But DO use “the” once it’s specific: “**the** data **we migrated last night**”, “**the** tests **in this PR**”.

The mnemonic: *Specific & known → the. New / countable & general → a/an. Mass noun or general plural → nothing.*

In your daily writing

- **Commit messages** are the one place where dropping articles is OK style: `fix null check in cart service` is fine. The imperative-log voice is accepted.
- **PR descriptions, Slack, code comments, docs** are real sentences — put the articles back: “This **PR** adds **a** retry to **the** webhook handler.”
- **Code comments:** “// returns **the** cached store, or null on **a** miss” reads better than “returns cached store or null on miss”.
- **Standups:** “I opened **a** PR for **the** login bug and started **research** on **the** caching layer.” (research = no article; the caching layer = the specific one.)

Drills

Fix or fill each one. Don’t peek — answers are below.

1. Fix: `can u do reaseach on grammer for me i already know but as of now i am making so much mistake`
2. Fill: I opened ____ PR and merged it into ____ main branch.
3. Choose: It throws (a / an) error when (the / a) database is down.
4. Fix: please review pull request before standup
5. Choose: This endpoint returns (a / an) HTTP 500 if (the / a) token expires.

6. Fill: We don't store ____ feedback in ____ database; we send it to ____ Slack.
 7. Fix: i fixed bug but tests still failing on main
 8. Choose: Reading the config takes (a / an) hour because (the / a) file is huge.
 9. Fill: Write ____ unit test for ____ function you just changed.
 10. Fix: this is honest mistake, i dropped article again
 11. Choose: We use (a / an) SQL query and (a / an) URL helper here.
 12. Fix: data is missing so much field after the migration
-

Answer key

1. **"Can you do research on grammar for me? I already know it, but right now I'm making so many mistakes."** — **you** not **u**, spell *research* and *grammar*, capital **I**, full stop, and "so many mistakes" (countable → many + plural).
 2. **a / the** — first mention of the PR → **a**; the one **main** everyone knows → **the**.
 3. **an / the** — "error" sounds vowel-ish → **an**; the specific database in context → **the**.
 4. **"Please review the pull request before standup."** — capital first word, and a known PR being discussed → **the**.
 5. **an / the** — "HTTP" sounds like *aitch* (vowel sound) → **an**; the specific token → **the**.
 6. **no article / the / no article** — "feedback" uncountable → no article; "the database" = the specific one → **the**; "Slack" is a name → no article.
 7. **"I fixed the bug, but tests are still failing on main."** — capital **I**, the known bug → **the**, comma before "but", and general "tests" → no article.
 8. **an / the** — "hour" has a silent h, sounds like *our* → **an**; the specific config file → **the**.
 9. **a / the** — one of many tests → **a**; the specific function you changed → **the**.
 10. **"This is an honest mistake; I dropped an article again."** — silent h → **an honest**; capital **I**; "an article" (one of many).
 11. **an / a** — "SQL" sounds like *ess* (vowel) → **an**; "URL" sounds like *you* (consonant) → **a**.
 12. **"Data is missing so many fields after the migration."** — "data" stays no article; "so many fields" (countable → many + plural **-s**); "the migration" = the specific one.
-

Part of the Developer English course — see [00-index.md](#).

02 — Countable vs uncountable: much / many / few / less / fewer

Source: [readme/research/english/02-countable-uncountable-much-many.md](#)

IN SHORT — KEY TAKEAWAYS

This document teaches you when to say “many” and when to say “much” — and when to add an “s” to a word or leave it out. Getting this wrong is one of the most common English mistakes for non-native speakers, and it shows up every time you write a message, email, or comment.

The main parts explained simply:

- **Countable nouns** — Things you can count one by one, like bugs, files, or orders. These words take “many,” “few,” and “fewer,” and they get an “-s” at the end when there is more than one. Example: “many bugs,” “fewer tickets.”
- **Uncountable nouns** — Things you measure, not count, like code, feedback, work, or time. These words take “much,” “little,” and “less,” and they never get an “-s.” Example: “much code,” “less work,” “some feedback” (never “feedbacks”).
- **The quick test** — Before you pick “much” or “many,” ask: can I put a number in front of it? “Three bugs” works, so “bug” is countable → use “many.” “Three codes” sounds wrong, so “code” is uncountable → use “much.”
- **“fewer” vs “less”** — Use “fewer” when the noun ends in “-s” (countable). Use “less” for everything else (uncountable). “Fewer mistakes, less time.” Code reviewers notice this one.
- **“a few” vs “few” (and “a little” vs “little”)** — “A few” means some, which is fine. “Few” means almost none, which sounds like a complaint. Same idea for “a little” vs “little.”
- **Common mistakes fixed** — The document walks through real examples like “so much mistake” (should be “so many mistakes”) and “less commits” (should be “fewer commits”), with a table showing what went wrong and why.
- **Drills and answers** — Twelve practice sentences with full answer explanations, so you can test yourself and learn from your mistakes right away.

What to do with this: Run the number test every time you write “much” or “many” — if the noun can be counted, switch to “many” and add “-s.” Learn these three pairs by heart: many/much, fewer/less, a few/a little.

What this fixes for you: you wrote “so much mistake” when you meant “so many mistakes.” This doc makes that choice automatic, so your commits, PRs, and standups stop leaking that one tell.

The rule in 30 seconds

- **Countable** nouns are things you can count one-by-one: one bug, two bugs, three commits. They take a plural -s and pair with **many / few / fewer / a number of**.

- **Uncountable** nouns are stuff you measure, not count: code, feedback, work, progress. They have **no plural -s** and pair with **much / little / less / an amount of**.
- Quick test: can you put a number in front of it? “Three bugs”  → countable. “Three codes”  → uncountable (say “three lines of code”).
- Your flagship fix: “**so much mistake**” → “**so many mistakes.**” A mistake is countable, so it needs **many** and the **-s**.

Why this trips you up

In your daily writing you reach for **much** by default and you drop the plural **-s** (“so much mistake,” “few bug left”). Many languages don’t mark countability the way English does, so the habit is invisible to you. The fix isn’t memorizing a list — it’s running the 30-second number test before you pick *much* or *many*.

See it / fix it

 What you tend to write	 Correct	Why
i am making so much mistake	I'm making so many mistakes.	<i>Mistake</i> is countable → many + plural -s . (Also: capital <i>I</i> , full stop.)
there is too much bug in this build	There are too many bugs in this build.	Countable → many bugs ; plural subject takes are .
i wrote so much code today	I wrote so much code today.	<i>Code</i> is uncountable → much is correct, no -s . (Just fix the lowercase <i>i</i> .)
got few feedbacks on the PR	I got some feedback on the PR.	<i>Feedback</i> is uncountable → no -s , and use a little / some , not few .
less commits this sprint	Fewer commits this sprint.	Countable → fewer , not less .
we have much open tickets	We have many open tickets.	Countable → many , not much .
there is many duplicate code	There is a lot of duplicate code.	<i>Code</i> is uncountable → a lot of / much , not many ; verb stays is .
i did a lot of researches on grammer	I did a lot of research on grammar.	<i>Research</i> is uncountable (no -s); spelling: <i>research</i> , <i>grammar</i> .
only little tests are failing	Only a few tests are failing.	Countable → a few ; little is for uncountable.
how much files did u change	How many files did you change?	Countable → many ; spell out <i>you</i> ; it's a question → ? .

Patterns to remember

The pairing table — never mix the columns:

Countable (count them)	Uncountable (measure it)
many bugs	much code
few / a few tests	little / a little time
fewer commits	less work
a number of files	an amount of data
too many tickets	too much feedback

Mnemonic: *Many = manys* → things you can count get the plural -s. *Much = mush* → stuff you can't pull apart.

few vs **a few** (and **little** vs **a little**):

- *few / little* = almost none, a complaint. “Few tests cover this.” (bad)
- *a few / a little* = some, a positive amount. “A few tests cover this.” (fine)

less vs **fewer** (the one reviewers notice):

- **fewer** + countable plural: “fewer merge conflicts.”
- **less** + uncountable: “less downtime.”
- Cheat: if the noun ends in **-s**, you almost always want **fewer**.

In your daily writing

- **Commit messages:** “Fix many flaky tests” not “Fix much flaky test.” “Remove unused code” (uncountable, no -s).
- **PR titles:** “Resolve fewer-than-expected conflicts” — and “Reduce duplicate code,” never “reduce duplicate codes.”
- **Standups (Slack):** “I closed three bugs and got some feedback on the design.” Count the bugs, measure the feedback.
- **Code review comments:** “There are too many responsibilities in this class” (countable) vs “This adds too much complexity” (uncountable).
- **Bug reports:** “Steps produce many errors in the console” — errors are countable, so plural **-s**.

A fast self-check before you hit send: find every *much / many / few / little / less / fewer*, then ask “**can I count this noun?**” If yes, you want *many / few / fewer* + **-s**. If no, you want *much / little / less*.

Drills

Fix, fill, or choose. No answers here — scroll to the key after you try.

1. Fix: `i am making so much mistake in my commit message`
2. Fix: `there is too much bug in the login flow`
3. Choose: We need (less / fewer) database queries on this page.

4. Fill: I got a lot of _____ (feedback / feedbacks) on the pull request.
 5. Choose: How (much / many) files does this migration touch?
 6. Fix: **we wrote less tests this sprint than last sprint**
 7. Fill: There is too _____ (much / many) duplicate code in this module.
 8. Choose: Only (a little / a few) edge cases are still failing.
 9. Fix: **i did a lot of researches on grammar before the refactor**
 10. Fill: This release ships with _____ (fewer / less) bugs and _____ (fewer / less) downtime.
 11. Choose: The API returns (much / a lot of) data, so paginate it.
 12. Fix: **how much commits are in this PR**
-

Answer key

1. **I'm making so many mistakes in my commit messages.** — *mistake* is countable → **many** + -s; capital *I*; messages plural too.
2. **There are too many bugs in the login flow.** — countable → **many bugs**; plural subject → **are**.
3. **fewer** — *queries* is countable (ends in -s) → **fewer**, not *less*.
4. **feedback** — uncountable → no -s.
5. **many** — *files* is countable → **many**.
6. **We wrote fewer tests this sprint than last sprint.** — *tests* is countable → **fewer**, not *less*; capital *W*.
7. **much** — *code* is uncountable → **much**.
8. **a few** — *edge cases* is countable → **a few**, not *a little*.
9. **I did a lot of research on grammar before the refactor.** — *research* is uncountable (no -s); spelling *research* and *grammar*; capital *I*.
10. **fewer** bugs and **less** downtime. — *bugs* countable → **fewer**; *downtime* uncountable → **less**.
11. **a lot of** — both work, but *much data* sounds formal; **a lot of data** is the natural dev phrasing. (Never *many data*.)
12. **How many commits are in this PR?** — *commits* is countable → **many**; question → **?**.

Part of the Developer English course — see [00-index.md](#).

03 — Subject–verb agreement

Source: [readme/research/english/03-subject-verb-agreement.md](#)

IN SHORT — KEY TAKEAWAYS

This document is about making your verb match your subject — the “doer” of the action in a sentence. In English, a single thing needs one form of the verb, and many things need a different form. Getting this right makes your writing look careful and professional, even in quick Slack messages or commit notes.

The main parts explained simply:

- **The basic rule** — One thing uses “is / does / returns / passes.” Many things use “are / do / return / pass.” The tricky part: always look for the *real* doer, not just the nearest word.
- **Hidden subjects — the “of ...” trap** — A phrase like “the list of files” might tempt you to write “the files are,” but the real subject is “list,” so the correct form is “the list is.” Cross out the “of ...” part to find the true subject.
- **Words that are always singular** — “Each,” “every,” “everyone,” “someone,” “nobody,” and “one of ...” always take the singular form, even when they feel like they refer to many people or things. Example: “Each of the services **needs** a restart.”
- **“There is” vs “There are”** — Use “there is” for one thing and “there are” for more than one. Match the verb to what comes right after it.
- **Team / group / list (collective nouns)** — In everyday professional English, treat these as one single thing. Write “The team **is** ready,” not “The team are ready.”
- **“Many” vs “much”** — Use “many” with things you can count (many bugs, many mistakes). Use “much” with things you cannot count (much progress, much memory). This also affects whether your noun needs a plural “-s.”
- **Everyday examples** — The document shows exactly how these rules apply to commit messages, pull-request titles, standup updates, code comments, and bug reports — the writing you do every day as a developer.
- **Drills and answer key** — Twelve practice sentences let you test yourself, with full corrections and plain explanations so you can see the pattern, not just memorise the answer.

What to do with this:

Before you send a message or write a commit, quickly ask: “Is my subject one thing or many?” Then pick the matching verb form. Pay special attention to “each / every / one of ...” — these always take the singular form and are the most common source of mistakes.

What this fixes for you: the verb matching the subject, so your PRs and standups stop saying “the tests passes” or “the users is” — small slips that make solid work read as sloppy.

The rule in 30 seconds

- One thing → singular verb. Many things → plural verb. “The test **passes**” / “The tests **pass**”.
- The verb agrees with the **real subject**, not the noun nearest to it: “The list of files **is** ready” (the *list* is ready, not the files).
- “each”, “every”, “everyone”, “one of ...” are all **singular**: “Each endpoint **returns** JSON”, “One of the tests **fails**”.
- “There is” for one, “There are” for many: “There **is** one failing test”, “There **are** two open PRs”.

Why this trips you up

In your first language the verb may not change with the subject the way it does in English, so you reach for one form and stick with it. You also tend to drop the plural **-s** on nouns (“so much mistake” instead of “so many mistakes”), which then knocks the verb out of agreement too. And when a long phrase sits between the subject and the verb (“the list of failing files”), you match the verb to whatever word is closest instead of the true subject.

See it / fix it

✗ What you tend to write	✓ Correct	Why
the tests passes now	the tests pass now	Plural subject ("tests") → plural verb (no -s).
the test pass after my fix	the test passes after my fix	Singular subject ("test") → verb takes -s.
the list of files are uploaded	the list of files is uploaded	The subject is "list" (singular), not "files".
each of the services need restart	each of the services needs a restart	"each" is singular → "needs"; also add the article "a".
there is two failing checks	there are two failing checks	Two checks → plural → "there are".
one of the migrations fail on prod	one of the migrations fails on prod	"one" is the subject → singular → "fails".
the API return 500 sometimes	the API returns 500 sometimes	Singular subject "API" → "returns".
the users is logged out randomly	the users are logged out randomly	Plural subject "users" → "are".
everyone have access to the repo	everyone has access to the repo	"everyone" is singular → "has".
the data are missing in the response	the data is missing in the response	In dev/everyday writing, treat "data" as singular: "data is".
so much mistake in this PR	so many mistakes in this PR	"mistakes" is countable → use "many" + plural -s.
none of the endpoints is documented	none of the endpoints are documented	With a plural "of" phrase, "none ... are" reads naturally.

Patterns to remember

Find the real subject first. Cross out the "of ..." phrase, then pick the verb.

Subject phrase	Real subject	Verb
the list of files	list	is / was
a set of rules	set	applies
one of the tests	one	fails
each of the configs	each	needs
the array of users	array	contains

Singular crew (always a singular verb): each, every, everyone, everybody, someone, nobody, anything, one of, neither (alone).

Each / every / -one / -body → **verb + s**. “Everybody **is** here”, “Each field **is** required”.

There is vs there are: match the verb to what comes *after* it.

“There **is** a bug.” / “There **are** bugs.” When listing mixed items, match the first: “There **is** one error and three warnings.”

Collective nouns (team, group, staff, list): in American English, treat them as **singular** by default. “The team **is** shipping today”, “The QA team **owns** this.” (British writing allows “the team are”, but stay singular to be safe.)

Countable test — many vs much:

Can you count it? → plural -s + **many** (many mistakes, many bugs, many requests). Can’t count it (a mass)? → **much** (much progress, much memory, much traffic).

In your daily writing

- **Commit messages:** “Fix race condition where the worker **drops** events” — singular “worker” → “drops”. Quick scan: is the doer one or many?
- **PR titles:** “These changes **make** the build **pass**” — plural “changes” → “make”; “build” is one → “pass” (because the helper “make” already carries the agreement, the next verb stays base form).
- **Standups (Slack):** “All the tests **pass** locally” not “passes”. “One of my branches **is** still red” not “are”.
- **Code comments:** “// This function **returns** null when the cache **is** empty” — both subjects singular → both verbs take -s/“is”.
- **Bug reports:** “The API **returns** 200 but the body **is** empty” — describe each singular subject with a singular verb; it reads precise and credible.

Drills

Fix the agreement (and any dropped plural / article you spot).

1. the list of failing tests are getting longer.
2. each of the services run in its own container.
3. one of the env variables are missing on staging.
4. there is three open pull request on this repo.
5. the API endpoints returns different shapes — fix this.
6. everyone on the team have merge rights now.
7. the data from the webhook are not validated.
8. Choose A or B: “Neither of the builds (A) pass / (B) passes the lint step.”

9. Fill the blank: "The set of rules ____ (apply) only to admin users."
10. so much edge case is not covered by the test.
11. Choose A or B: "There (A) is / (B) are a memory leak and two slow queries."
12. the team are responsible for this incident, and i will write the postmortem.
-

Answer key

1. **The list of failing tests is getting longer.** Subject is "list" (singular) → "is"; also capitalize the first word.
 2. **Each of the services runs in its own container.** "each" is singular → "runs".
 3. **One of the env variables is missing on staging.** "one" is the subject → "is".
 4. **There are three open pull requests on this repo.** Three → "are"; "requests" needs the plural -s.
 5. **The API endpoints return different shapes — fix this.** Plural "endpoints" → "return" (no -s).
 6. **Everyone on the team has merge rights now.** "everyone" is singular → "has".
 7. **The data from the webhook is not validated.** Treat "data" as singular in dev writing → "is".
 8. **B — passes.** "Neither" alone is singular → "passes".
 9. **applies.** "The set ... applies"; subject is "set" (singular), not "rules".
 10. **So many edge cases are not covered by the test.** Countable → "many" + plural "cases" → "are"; capitalize "So".
 11. **A — is.** "There is" matches the first item ("a memory leak", singular).
 12. **The team is responsible for this incident, and I will write the postmortem.** American English: collective "team" is singular → "is"; capitalize "I".
-

Part of the Developer English course — see [00-index.md](#).

04 — Verb tenses & keeping them consistent

Source: [readme/research/english/04-verb-tenses.md](#)

IN SHORT — KEY TAKEAWAYS

This document teaches you which verb form to use depending on when something happened — and how to stay consistent within a sentence. If you have ever written “I am fix the bug” or mixed up “I fixed” with “I have fixed”, this is the fix. Getting tenses right makes your standups, commit messages, and PR descriptions sound clear and professional.

The main parts explained simply:

- **Present simple** — Use this to describe how code works or a regular habit. Example: “The function returns a UUID.” or “We deploy on Fridays.” This is your default tense for code comments and documentation.
- **Past simple** — Use this when something is fully finished and you know exactly when. Example: “I fixed the bug yesterday.” If your sentence mentions a specific time (yesterday, last Monday, at 3pm), use this tense.
- **Present perfect (“have/has + done”)** — Use this when something just finished but it still matters right now. Example: “I have pushed the fix.” or “The build has finished.” Use words like “already”, “just”, or “so far” as a signal for this tense.
- **Present continuous (“am/is/are + doing”)** — Use this for something happening right now, in progress. Example: “I am working on it.” or “The tests are running.”
- **The “I am ____” trap** — After “am”, “is”, or “are”, you must add -ing to the next verb. “I am fix” is wrong. “I am fixing” is right. If you do not want -ing, drop the helper word: just say “I fixed it.”
- **Perfect vs. past: the time-word test** — Named time (yesterday, last week, on Monday) → use past simple. Vague relevance now (already, just, so far) → use present perfect.
- **Commit and PR titles use the command form** — Write “Fix the bug”, not “Fixed the bug” or “Fixing the bug.” Git titles complete the sentence: “If applied, this commit will Fix the bug.” Always the bare command: Fix, Add, Remove, Update.
- **Different places, different tenses** — Standup past = past simple. Standup today = present continuous. Code comments = present simple. Bug reports = past simple for what happened, present simple for what keeps happening.

What to do with this: Work through the 12 drills at the bottom of the document — they cover every trap listed above. Then check your next PR description and standup message against the four-tense cheat sheet before you send them.

What this fixes for you: you'll stop writing “I am fix the bug”, you'll pick the right tense for commits and standups, and your sentences will stop jumping between past and present halfway through.

The rule in 30 seconds

- **Present simple** = states, habits, and how code works. *"The function returns a UUID."* *"We deploy on Fridays."*
- **Past simple** = a finished action at a finished time. *"I fixed the bug yesterday."* *"The build failed last night."*
- **Present perfect** (**have/has** + past participle) = done, and it still matters now. *"I have pushed the fix."* *"The build has finished."*
- **Present continuous** (**am/is/are** + verb-**ing**) = happening right now, in progress. *"I am working on it."* *"The tests are running."*
- Stay in **one tense** per thought. Don't start in the past and drift into the present in the same sentence.

Why this trips you up

You reach for "I am" and then attach a plain verb — "I am fix" — but **am/is/are** must be followed by an -**ing** verb (**am fixing**) or you should drop the helper and use past simple (**I fixed**). You also slide between tenses inside one sentence because, in the moment of writing a standup or PR, the timeline in your head shifts and the verbs follow it. And small slips pile up — a lowercase "i", a missing full stop, "u" for "you" — which make even correct tenses read as rushed.

See it / fix it

✗ What you tend to write	✓ Correct	Why
i am fix the login bug now	I am fixing the login bug now.	<code>am</code> needs an -ing verb; also capitalize “I” and the sentence start, and end with a full stop.
yesterday i am fixing the bug	Yesterday I fixed the bug.	“Yesterday” is finished time → past simple, not present continuous.
I have pushed the fix yesterday	I pushed the fix yesterday.	Present perfect can’t take a finished-time word like “yesterday”; use past simple.
the build finish, you can deploy	The build has finished, so you can deploy.	“Just done, matters now” → present perfect (<code>has finished</code>); add the article/connector and full stop.
this function return a token	This function returns a token.	How code works = present simple; third person singular needs -s (<code>returns</code>).
i already fix it but i make so much mistake	I already fixed it, but I’m making so many mistakes.	Past simple for the done action; many + plural mistakes (countable); fix “i”.
Fixed the typo in readme (PR title)	Fix the typo in README (PR title / commit).	Commit/PR titles use the imperative (“Fix”), not past tense.
we are deploy every friday	We deploy every Friday.	Habit/schedule = present simple, not continuous; capitalize “Friday”.
can u do reaseach on grammer for me	Can you do research on grammar for me?	Spelling: research, grammar; “u” → “you”; it’s a question → “?”.
open pull request and i review it	Open a pull request and I’ll review it.	Missing article “a”; capitalize “I”; “I’ll” (future) reads cleaner here.

Patterns to remember

The four workhorse tenses, as a cheat sheet:

Tense	Shape	Use it for	Example
Present simple	<code>verb</code> (+s)	how code works, habits, schedules	“The API caches the response.”
Past simple	<code>verb-ed</code> / irregular	finished action, finished time	“I reverted the commit.”
Present perfect	<code>have/has</code> + participle	just done / still relevant now	“I have merged the branch.”
Present continuous	<code>am/is/are</code> + <code>verb-ing</code>	in progress right now	“CI is running .”

The “I am ____” trap: after `am / is / are`, the next verb almost always ends in **-ing**.

- ❌ I am fix → ✅ I am **fixing**
- ❌ It is run → ✅ It is **running**
- If you don't want -ing, drop the helper: "I **fixed** it."

Perfect vs. past — the time-word test: if the sentence names a finished time (*yesterday, last week, at 3pm, on Monday*), use **past simple**. If it's *already / just / so far / now*, use **present perfect**.

- "I **deployed** it on Monday." (named time → past)
- "I've **already** deployed it." (relevance now → perfect)

Commits & PR titles are imperative on purpose. Git wants the title to complete the sentence "*If applied, this commit will ____*". So write the bare command form: **Fix, Add, Remove, Refactor** — never "Fixed", "Adds", or "Fixing".

- ✅ Add rate limiting to login endpoint
- ✅ Fix null pointer in cart total
- ❌ Added rate limiting / ❌ Fixing login / ❌ Adds endpoint

In your daily writing

- **Commit messages / PR titles:** imperative mood. "Fix", "Add", "Update", "Remove". One line, capital first letter, no full stop needed on the subject line.
- **PR description (body):** present perfect or past for what you did — "I've moved the validation into the service layer" or "Moved validation into the service layer." Present simple for what the code now does — "The service now rejects empty payloads."
- **Standup (Slack):** three clean tenses. *Yesterday* → past simple ("I fixed the webhook retry"). *Today* → present continuous ("I'm writing tests for it"). *Blockers* → present simple ("I need access to the staging DB").
- **Code comments / docs:** present simple, because they describe how the code behaves every time it runs. "Returns the cached value if present." Not "Returned" or "Will return".
- **Bug reports:** past simple for what happened ("The page crashed when I clicked Save"), present simple for the steady behavior ("The error appears on every submit").

Drills

For each, fix the sentence, fill the blank, or choose A/B. Write professional dev English — capitals, full stops, real spelling.

1. Fix: i am fix the failing test right now
2. Fix: yesterday i have deployed the new version
3. Fill the blank (commit title): ____ pagination to the orders list endpoint (choose: Added / Add / Adding)
4. Choose A or B: This function (A) *return* / (B) *returns* the user's UUID.

5. Fix: `the migration finish so u can run the seeder`
 6. Choose A or B: Right now I (A) *work* / (B) *am working* on the checkout flow.
 7. Fix: `i already fix the bug but i still make so much mistake in the tests`
 8. Fill the blank: I ____ (just push) the hotfix, can you review it? (use the right tense)
 9. Choose A or B: We (A) *are releasing* / (B) *release* every two weeks, like clockwork.
 10. Fix (PR title): `Fixed the race condition in the queue worker`
 11. Fix the run-on: `the build broke i looked at the logs the error was a missing env var i added it now its green`
 12. Fill the blank: The endpoint ____ (cache) the response for 60 seconds. (how the code works — pick the tense)
-

Answer key

1. **I am fixing the failing test right now.** — After “I am”, the verb takes **-ing** (`fixing`); capitalize “I” and the start, add a full stop.
2. **Yesterday I deployed the new version.** — “Yesterday” is finished time, so use **past simple**, not present perfect.
3. **Add pagination...** — Commit titles use the **imperative**: “Add”, not “Added” or “Adding”.
4. **B — returns.** — How the code works = present simple, and third person singular adds **-s**.
5. **The migration has finished, so you can run the seeder.** — Just done / relevant now → present perfect (`has finished`); “u” → “you”; add article-less connector “so” and a full stop.
6. **B — am working.** — “Right now” + in progress = present continuous.
7. **I already fixed the bug, but I still make so many mistakes in the tests.** — Past simple for the done fix; **many** + plural **mistakes** (countable); fix “i”.
8. **I’ve just pushed** the hotfix... — “Just” + still relevant = present perfect (`have/’ve` + past participle `pushed`).
9. **B — release.** — A regular schedule/habit = present simple.
10. **Fix the race condition in the queue worker.** — PR/commit titles use the **imperative** (“Fix”), not past tense.
11. **The build broke. I looked at the logs, and the error was a missing env var. I added it, and now it’s green.** — Split the run-on into sentences with full stops/commas; keep past simple for the finished steps; “its” → “it’s” (it is).
12. **caches** — Steady behavior of the code = present simple; third person singular adds **-s** (`caches`).

Part of the Developer English course — see [00-index.md](#).

05 — Plurals, apostrophes & possessives (its vs it's)

Source: [readme/research/english/05-plurals-and-possessives.md](#)

IN SHORT — KEY TAKEAWAYS

This document teaches you three small but very common English mistakes: when to add an apostrophe (the `'` mark), when NOT to add one, and how to use “its” vs “it’s” correctly. These mistakes appear in commit messages, PR titles, and Slack messages every day, and fixing them makes your writing look much more professional.

The main parts explained simply:

- **Plain plurals — never use an apostrophe** — When you just mean “more than one thing,” add `-s` or `-es` and nothing else. Write `PRs`, `APIs`, `bugs`, `tickets` — never `PR's`, `API's`, `bug's`. The apostrophe is not needed and is wrong here.
- **Possessives — use apostrophe + s** — When something *belongs to* a person or thing, add `'s`. Example: “the server’s logs” means the logs belong to the server. “the team’s decision” means the decision belongs to the team.
- **Plural possessives — apostrophe after the s** — When something belongs to *many* people or things (and the word already ends in `-s`), just add an apostrophe after: “the developers’ commits” means commits belonging to several developers.
- **its vs it’s — the unzip test** — `it's` (with apostrophe) always means “it is” or “it has.” If you can replace it with “it is” and it still makes sense, use `it's`. If not, use `its` (no apostrophe). Example: “It’s broken” = “It is broken” (correct). “Its config” = “It is config” (does not make sense, so use `its`).
- **Plurals spelling guide** — Words ending in `ch`, `sh`, `x`, `z`, or `s` add `-es` (branch → branches, class → classes). Words ending in consonant + `y` change to `-ies` (query → queries). Acronyms just get `-s` (API → APIs, ID → IDs).
- **Irregular plurals** — A few common tech words follow no rule: index → indexes, schema → schemas, datum → data. These are worth memorising separately.
- **Daily writing examples** — The document shows how these rules apply in commit messages, PR titles, Slack standups, code comments, and bug reports, so you can practise in real situations.

What to do with this:

Before you use an apostrophe, ask yourself: “Am I saying something *belongs to* someone?” If not, leave it out. And whenever you write “its” or “it’s,” try to expand it to “it is” — if that works, add the apostrophe; if not, leave it off.

What this fixes for you: you’ll stop writing “API’s” for plain plurals, you’ll get “the server’s logs” vs “the developers’ commits” right, and you’ll never again mix up **its** and **it’s** in a commit message or PR.

The rule in 30 seconds

- **Plain plural** = just add **-s** (or **-es**). More than one thing. No apostrophe ever: `APIs`, `PRs`, `repos`, `bugs`, `IDs`.
- **Possessive** = **apostrophe + s**. Something belongs to someone: `the server's logs`, `the team's decision`, `Pravin's branch`.
- **Plural + possessive** = **-s first, then just an apostrophe**: `the developers' commits` (commits of many developers).
- **it's** = “it is” / “it has”. **its** (no apostrophe) = belongs to it. If you can't expand it to “it is”, use **its**.

Why this trips you up

You write commits, PRs, and Slack messages fast, so apostrophes get sprinkled on autopilot — “API's are down” feels natural but is wrong. As a non-native speaker you also sometimes drop the plural **-s** entirely (“so much mistake” instead of “so many mistakes”), which hides the difference between one and many. And **its** vs **it's** is a coin-flip right now because both sound identical out loud.

See it / fix it

✗ What you tend to write	✓ Correct	Why
<code>merged 3 PR's today</code>	<code>merged 3 PRs today</code>	Plain plural of an acronym — no apostrophe.
<code>the API's return JSON</code> (meaning many APIs)	<code>the APIs return JSON</code>	“More than one API” is a plural, not a possessive.
<code>its failing in prod</code>	<code>it's failing in prod</code>	“It is failing” → use it's .
<code>the service lost it's connection</code>	<code>the service lost its connection</code>	The connection belongs to it → its (no apostrophe).
<code>the developers commits are messy</code>	<code>the developers' commits are messy</code>	Commits of many developers → plural possessive, apostrophe after the -s .
<code>i fixed so much bug in this PR</code>	<code>I fixed so many bugs in this PR</code>	Bugs are countable → many + plural -s ; also capital I .
<code>the server logs is too noisy</code>	<code>the server's logs are too noisy</code>	Logs belong to the server → <code>server's</code> ; logs is plural → are .
<code>closed all the ticket's for this sprint</code>	<code>closed all the tickets for this sprint</code>	Plain plural — drop the apostrophe.
<code>the team decision was to revert</code>	<code>the team's decision was to revert</code>	The decision belongs to the team → possessive.
<code>we deployed in the 2020's</code>	<code>we deployed in the 2020s</code>	Plural of a number/decade — no apostrophe.

Patterns to remember

The apostrophe test (run this every time):

1. Am I just talking about *more than one*? → **no apostrophe**. `repos` , `PRs` , `IDs` , `URLs` .
2. Does something *belong to* one thing? → **'s** . `the bug's root cause` , `Redis's config` .
3. Does it belong to *many* things already ending in `-s` ? → **just ' .** `the users' sessions` , `the developers' commits` .

Plurals cheat sheet:

Word ends in...	Add	Example
most words	<code>-s</code>	bug → bugs, repo → repos
<code>s</code> , <code>x</code> , <code>z</code> , <code>ch</code> , <code>sh</code>	<code>-es</code>	branch → branches, patch → patches, class → classes
consonant + <code>y</code>	<code>-ies</code>	query → queries, dependency → dependencies
acronym / number	<code>-s</code> only	API → APIs, ID → IDs, the 90s

Common irregulars (no `-s` rule):

Singular	Plural
index	indexes / indices
matrix	matrices
schema	schemas (sometimes schemata)
datum	data (treat “data” as plural or as a mass noun)
person	people
child	children

its vs it's — the only mnemonic you need:

`it's` always unzips into “**it is**” or “**it has**”. If it won't unzip, it's `its` . “It's broken” → “It is broken”
✓. “Its README” → “It is README”? ✗ → so use `its` .

(Same trick works for `you're` / `your` and `they're` / `their` — the apostrophe version always expands.)

In your daily writing

- **Commit messages:** `Fix the API's rate limiter` only if you mean *one* API's limiter. For many: `Fix the APIs' rate limiters` . For a plain count: `Update 4 endpoints` .

- **PR titles:** `Refactor the worker's retry logic` (one worker) vs `Standardize the workers' retry logic` (all of them). Never `worker's` when you just mean several workers.
- **Slack standups:** “All the tests are green” — not “test’s”. “It’s deployed” (it is) vs “its config changed” (the config belonging to it).
- **Code comments:** `// the cache stores its own TTL` — possessive, no apostrophe. A wrong `it's` here survives forever in the codebase.
- **Bug reports:** “The endpoint returns 500s” (plural of a number, no apostrophe). “The server’s response is malformed” (possessive).

Drills

Fix the sentence, fill the blank, or choose A or B. (Answers below — don’t peek.)

1. Fix: `i merged 5 PR's and closed 3 ticket's today`
 2. Choose: The build failed because (A) **its** (B) **it's** missing a dependency.
 3. Choose: We should cache (A) **its** (B) **it's** response to cut latency.
 4. Fill the blank: All three _____ (microservice) share one database.
 5. Fix: `the developers commits broke the pipeline` (you mean several developers)
 6. Fix: `i fixed so much typo in the docs`
 7. Choose: The queue lost (A) **its** (B) **it's** ordering after the restart.
 8. Fill the blank: We migrated all the _____ (legacy API) last quarter.
 9. Fix: `the server log's are full of 500's`
 10. Choose: (A) **Its** (B) **It's** been flaky since the 2020s.
 11. Fix: `the team decision is final and the user's data is safe` (the decision belongs to the team; data belongs to many users)
 12. Fill the blank: I ran the _____ (query) against two _____ (index).
-

Answer key

1. **I merged 5 PRs and closed 3 tickets today.** — Capital **I**; plain plurals take no apostrophe.
2. **B — it's.** “It is missing a dependency” expands cleanly, so use **it's**.
3. **A — its.** The response belongs to it → possessive **its**, no apostrophe.
4. **microservices.** Plural of a regular noun — add `-s`.
5. **The developers’ commits broke the pipeline.** — Several developers → plural possessive, apostrophe *after* the `-s`.
6. **I fixed so many typos in the docs.** — Typos are countable → **many** + plural `-s`; capital **I**.
7. **A — its.** “The queue lost it is ordering”? No — the ordering belongs to it → **its**.
8. **legacy APIs.** Acronym plural — just `-s`, never `API's`.

9. **The server's logs are full of 500s.** — Logs belong to the server (`server's`); `logs` is plural so use **are**; plural of a number (`500s`) has no apostrophe.
10. **B — It's.** “It has been flaky” / “It is” expands → **it's**; and `2020s` (decade plural) takes no apostrophe.
11. **The team's decision is final and the users' data is safe.** — `team's` (one team owns it); `users'` (data belonging to many users → apostrophe after the `-s`).
12. **queries / indexes.** “query” → `-ies` (consonant + `y`); “index” → **indexes** (or *indices*) is irregular.
-

Part of the Developer English course — see [00-index.md](#).

06 — Prepositions (in, on, at, to, for, with)

Source: readme/research/english/06-prepositions.md

IN SHORT — KEY TAKEAWAYS

This document is about the small connecting words called prepositions — words like “in,” “on,” “at,” “to,” and “for.” These tiny words cause big mistakes because there is often no logic to them; you just have to learn which word goes with which verb. This guide gives you simple rules and a cheat table so you can stop guessing.

The main parts explained simply:

- **The zoom rule (in / on / at)** — Think of it as going from big to small. Use “in” for large containers or long time periods (in the repo, in June). Use “on” for surfaces or specific days (on the branch, on Monday). Use “at” for exact points (at port 4000, at 9am). Each level has its own word.
- **Fixed pairs (collocations)** — Many verbs always go with one specific preposition. “Depend on,” “based on,” “responsible for,” “good at,” “interested in” — these never change. You cannot work them out from logic; you just memorize the whole pair as one unit.
- **Direction vs. location** — When something is moving somewhere, use “to” (deploy to production, push to remote, reply to a comment). When something is already sitting inside a place, use “in” or “on” (in the repo, on the branch).
- **Common mistakes table** — The document shows the exact errors that come up most often in daily developer writing — things like “depend to,” “based of,” “comment in my PR,” and “listens in port” — and shows the correct version with a short reason.
- **Daily writing examples** — The guide shows how to apply the rules in real situations: commit messages, PR titles, Slack standups, code comments, and bug reports.
- **Drills with answers** — There are 12 practice sentences to fix, with a full answer key. Each answer also explains the other mistake in the sentence (spelling, capital letters, punctuation).

What to do with this:

Bookmark the cheat table and check it whenever you write a verb you are unsure about. When you learn a new verb, always learn it together with its preposition as one pair — never the verb alone.

What this fixes for you: prepositions are the small words you keep guessing — “depend on” or “depend to”? This doc gives you a cheat table so you stop guessing and just know.

The rule in 30 seconds

- **Time/place** follow a zoom rule: **in** = big/inside (in June, in the repo), **on** = surface/day (on Monday, on the branch), **at** = exact point (at 9am, at line 42).
- Most dev prepositions are **fixed pairings** (collocations) — `depend on` , `based on` , `responsible for` . You **memorize** these; you do not work them out from logic.

- Direction uses **to**: `push to` , `deploy to` , `merge to` . Being inside something uses **in/on**: `in the repo` , `on the branch` .
- When in doubt, check the cheat table at the bottom. Real English speakers also just memorized these.

Why this trips you up

In your native language one preposition often covers several English ones, so you translate word-for-word and get “depend to” or “responsible of.” You also mix the zoom levels — writing “on the repo” when it’s “in the repo.” The fix is not grammar logic; it’s pairing the verb and its preposition together as one chunk and reusing it.

See it / fix it

✗ What you tend to write	✓ Correct	Why
this fix depend to the cache layer	This fix depends on the cache layer.	The verb is <code>depend on</code> , always. Also capitalize the first word and add the full stop.
i merged the branch to main	I merged the branch into main. (or to main)	You merge code into a target. Capitalize I .
pushed my changes in the remote	Pushed my changes to the remote.	Pushing is direction → to , not <code>in</code> .
the price is based of the tier	The price is based on the tier.	Fixed pair <code>based on</code> . Never <code>based of</code> / <code>based off</code> in formal writing.
who is responsible of this module	Who is responsible for this module?	<code>responsible for</code> . And it’s a question → add <code>?</code> .
she is good in debugging	She is good at debugging.	<code>good at</code> a skill. <code>in</code> is wrong here.
i am interested on the new ticket	I’m interested in the new ticket.	<code>interested in</code> , always. Capitalize I .
the server listen in port 4000	The server listens on port 4000.	A service listens on a port. Add the <code>-s</code> (it listens).
we will deploy in production tonight	We’ll deploy to production tonight.	Deploy is direction → to an environment.
can u comment in my PR	Can you comment on my PR?	You comment on something. Spell out <code>you</code> .
working in a ticket about grammer	Working on a ticket about grammar.	You work on a task. Fix the spelling: <code>grammar</code> .
the file is on the repo on branch dev	The file is in the repo on branch dev.	You’re in a repo (container) but on a branch.

Patterns to remember

Time & place — the zoom rule (big → small):

Word	Time	Place
in	in 2026, in June, in an hour	in the repo, in the file, in production
on	on Monday, on June 16, on the weekend	on the branch, on line 42, on a server
at	at 9am, at noon, at deploy time	at the endpoint, at the office

Dev collocation cheat table (memorize the pairing, not the logic):

Phrase	Preposition	Example
depend ____	on	The job depends on Redis.
based ____	on	Pricing is based on quantity.
responsible ____	for	I'm responsible for the queue worker.
good ____ (a skill)	at	He's good at writing tests.
interested ____	in	I'm interested in the caching task.
listen ____ (a port)	on	The service listens on port 8000.
deploy ____ (an env)	to	We deploy to staging first.
push ____ (a remote)	to	Push to origin.
merge ____ (a target)	into / to	Merge into main.
work ____ (a task)	on	I'm working on the bug.
comment ____ (a thing)	on	Please comment on the PR.
reply ____	to	Reply to the review thread.
connect ____	to	Connect to the database.
migrate ____	to	Migrate to PostgreSQL.
consist ____	of	The pipeline consists of three stages.

Quick mnemonic: Moving somewhere → **to** (push to, deploy to, reply to). Sitting inside → **in** (in the repo, in production). On a surface or a list → **on** (on the branch, on a port, on the PR).

In your daily writing

- Commits:** “Fix crash that **depends on** a null tenant” — not “depend to.”
- PR titles:** “Deploy worker **to** staging” / “Merge feature **into** main.”

- **Slack standups:** “Today I’m **working on** the cache ticket and will **comment on** Pravin’s PR.” — not “working in” / “comment in.”
- **Code comments:** `// Service listens on port 4000` — not “listens in port.”
- **Bug reports:** “The error happens **in** production **at** the checkout endpoint **on** the release branch.”
Notice each zoom level gets its own word.

Drills

Fix the preposition (and any spelling/capital/punctuation slips you notice).

1. the build depend to a env variable that is not set
2. i am interested on picking up the payment ticket
3. we deploy in production every friday
4. can u comment in my pull request when u get time
5. the price calculation is based of the product size
6. who is responsible of the queue worker
7. our api listen in port 8000 by default
8. she is really good in writing clean tests
9. Choose A or B: I’ll push my changes (A) **to** / (B) **in** the remote tonight.
10. Choose A or B: The config file is (A) **on** / (B) **in** the repo, (A) **on** / (B) **in** the dev branch.
11. Fill the blank: This migration consists ____ two reversible steps.
12. Fix the run-on: i fixed so much bug today and i will merge to main and then i deploy in staging

Answer key

1. **The build depends on an env variable that is not set.** — `depend on` ; add `-s` , capitalize, use `an env` , add full stop.
2. **I’m interested in picking up the payment ticket.** — `interested in` ; capitalize `I` .
3. **We deploy to production every Friday.** — Direction → `to` ; capitalize the day `Friday` .
4. **Can you comment on my pull request when you get time?** — `comment on` ; spell out `you` ; it’s a question → `?` .
5. **The price calculation is based on the product size.** — Fixed pair `based on` , not `based of` .
6. **Who is responsible for the queue worker?** — `responsible for` ; add the question mark.
7. **Our API listens on port 8000 by default.** — A service listens **on** a port; add `-s` .
8. **She is really good at writing clean tests.** — `good at` a skill, not `good in` .
9. **A — to .** — Pushing is direction, so `push to the remote` .
10. **B (in the repo), A (on the dev branch).** — You’re inside the repo but on a branch.
11. **of** — `consist of` . (The migration consists **of** two reversible steps.)

L2. I fixed so many bugs today. I'll merge into main and then deploy to staging. — so many bugs
(countable, plural); split the run-on into sentences; merge into ; deploy to ; capitalize I .

Part of the Developer English course — see [00-index.md](#).

07 — Punctuation & capitalization basics

Source: [readme/research/english/07-punctuation-and-capitalization.md](#)

IN SHORT — KEY TAKEAWAYS

This document teaches the small punctuation marks and capital letters that make written English look polished and professional. Most non-native writers skip these without realising it — but they are what separate a message that looks rushed from one that looks confident and senior.

The main parts explained simply:

- **Always write “I” as a capital** — The word for yourself is always a big letter “I”, never a small “i”. This one habit alone makes a huge difference to how your messages read.
- **Start every sentence with a capital letter** — The very first word of each sentence gets a big letter. If you forget this, sentences blur together and become hard to follow.
- **End every sentence with a mark** — Use a full stop (.) to finish a statement, a question mark (?) to finish a question, and an exclamation mark (!) only rarely for strong emphasis. Without these, the reader cannot tell where one idea stops and the next begins.
- **Capitalize product and tool names** — Names like GitHub, PostgreSQL, Slack, and Stripe have their own official spelling. Copy it exactly — never write them all in lowercase.
- **Commas give your sentences breathing room** — Use a comma to separate items in a list, after an opening phrase, and before words like “but”, “and”, or “so” that join two full sentences.
- **Colon vs full stop** — A colon (:) introduces a list or explanation. When in doubt, a full stop is always safe.
- **Backticks for code and commands** — Wrap any command, file name, or technical term in backtick marks so readers know it is code, not a spelling mistake.
- **Daily writing: commits, PRs, Slack, comments** — The document gives specific rules for each: commit titles start with a capital; PR descriptions use full sentences; Slack standups still need capital “I”; code comments are sentences too.
- **Practice drills with answers** — Twelve short exercises let you fix real mistakes and check your work immediately.

What to do with this:

Pick two habits to fix first: always write a capital “I” and always end each sentence with a full stop or question mark. These two changes alone will make your messages read as far more careful and professional.

What this fixes for you: the lowercase “i”, the missing full stops, and the run-on sentences that make your commits and Slack messages read as rushed. Small marks, big difference in how senior your writing looks.

The rule in 30 seconds

- Start every sentence with a **capital letter**, and always write **I** (the word for yourself) as a capital — never `i`.
- End every sentence with a **closing mark**: `.` for a statement, `?` for a question, `!` for strong emphasis (use `!` rarely at work).
- **Capitalize proper nouns** — names of people, products, and tools: `GitHub`, `Slack`, `PostgreSQL`, `Docker`, `Laravel`.
- Use **commas** to break up lists and clauses so a long sentence has air; use **backticks** around code so `prod` doesn't read as a typo.

Why this trips you up

You type the way you text — lowercase `i`, no full stops, and one long line where several sentences chain together with no breaks (your own message: *"i already know but as of now i am making so much mistake"*). In code that's fine, but in a PR or standup it makes a reader work to find where one thought ends and the next begins. The fix is mechanical, not creative: capital at the start, mark at the end, comma in the middle.

See it / fix it

✗ What you tend to write	✓ Correct	Why
can u do reaseach on grammer for me	Can you do research on grammar for me?	Capital start, real words not u / reaseach / grammer , and a ? because it's a question.
i already know but as of now i am making so much mistake	I already know it, but right now I'm making so many mistakes.	Capital I (twice), full stop, comma before but , and many mistakes (countable → plural).
fixed the bug it was a null pointer	Fixed the bug. It was a null pointer.	Two thoughts = two sentences. A full stop, then a new capital.
i deployed to prod but it failed i rolled back	I deployed to prod, but it failed, so I rolled back.	Capital I , commas to separate the chained clauses, no run-on.
can you review my pr when ur free	Can you review my PR when you're free?	PR is capitalized, ur → you're , and it ends with ? .
the api returns 500 check the logs	The API returns 500 – check the logs.	Capitalize the sentence and the proper-ish term API ; split the two ideas.
we use postgresql and github actions	We use PostgreSQL and GitHub Actions.	Product names keep their official capitalization.
to open pull request run gh pr create	To open a pull request, run `gh pr create`.	Add the article a , a comma after the intro clause, and backticks around the command.
done with task lgtm	Done with the task. LGTM!	Add the , end the statement, and LGTM (an acronym) stays uppcase.
i think we need three thing migration seeder and test	I think we need three things: a migration, a seeder, and a test.	Capital I , plural things , a colon to introduce the list, commas between items.

Patterns to remember

Capitalize when:

- It's the **first word** of a sentence.
- It's the word **I** (always, even mid-sentence).
- It's a **proper noun**: GitHub , Slack , PostgreSQL , Redis , Nuxt , Stripe , Monday , June .

Tool names are not yours to lowercase. It's GitHub , not github ; PostgreSQL , not postgresql .
When in doubt, copy the spelling from the project's own homepage.

End marks — pick one per sentence:

Sentence type	Mark	Example
Statement	.	The build passed.
Question	?	Did the build pass?
Strong call-out (rare at work)	!	Don't merge yet!

Comma — the four uses you actually need:

Use	Pattern	Example
Lists	a, b, and c	Add a test, a migration, and a seeder.
After an intro clause	When X, do Y.	After the deploy, watch the logs.
Before <code>but</code> / <code>and</code> / <code>so</code> joining two full sentences	S1, but S2.	It built, but the tests failed.
Around an aside	X, which is Y, ...	The cache, which is Redis, was stale.

Test for the “joining” comma: if both sides could stand alone as their own sentence, put a comma before `but` / `and` / `so`. `It built` ✓ + `the tests failed` ✓ → `It built, but the tests failed.`

Colon vs semicolon:

- **Colon** `:` introduces a list or an explanation. `We need three things: a migration, a seeder, and a test.`
- **Semicolon** `;` joins two closely related full sentences without `and` / `but`. `The deploy finished; the alerts cleared.` (When unsure, just use a full stop — it’s never wrong.)

Quotes and code:

- Wrap commands, file names, branches, and values in **backticks**: `run \ npm run build``, the ``main`` branch, set ``status`` to ``paid``.
- Use “quotes” for actual spoken/written phrases or UI labels: `Click "Save changes".`

In your daily writing

- **Commit messages:** capital first word, no full stop on the short summary line (git convention), backticks in the body. `Fix null pointer in OrderService` then a blank line, then `The \ customer` relation was null after a soft delete.``
- **PR titles & descriptions:** title is capitalized; the description is full sentences with end marks. End any direct question to reviewers with `? — Should I split this into two PRs?`

- **Slack standups:** still capitalize `I` and start sentences with a capital. `Yesterday I shipped the cache fix. Today I'm on the migration. No blockers.` reads far better than `yesterday i shipped... today i...`
- **Code comments:** capitalize the first word and end with a full stop. `// Guard against a null tenant after deletion.` — a comment is a sentence, treat it like one.

Drills

Fix, fill, or choose. Answers are at the bottom — try first, then check.

1. Fix: `i pushed the branch can u check it`
 2. Fix: `we use postgresql in dev and sqlite in test`
 3. Fix: `the deploy failed i checked the logs it was a config issue`
 4. Choose A or B: `It built ( A: , but / B: but ) the tests failed.`
 5. Fill the blank with `:` or `;` — `We need three things___ a test, a migration, and a seeder.`
 6. Fix: `can you open pull request for this`
 7. Fix: `i think we have so much bug in this release`
 8. Choose A or B: `After the deploy ( A: watch / B: , watch ) the error rate.`
 9. Fix: `done lgtn ship it`
 10. Fix: `the api returns 500 when ur token is expired`
 11. Fill the blank with the right end mark — `Did the migration run on staging___`
 12. Fix: `i did reaseach on grammer like u asked`
-

Answer key

1. `I pushed the branch. Can you check it?` — Capital `I`, split into two sentences, `u` → `you`, and the second is a question (`?`).
2. `We use PostgreSQL in dev and SQLite in test.` — Capitalize the sentence and the product names (`PostgreSQL`, `SQLite`); add the full stop.
3. `The deploy failed. I checked the logs – it was a config issue.` — Three thoughts; don't chain them. Capital start, capital `I`, end mark. (A dash or full stop both work for the last break.)
4. `A — It built, but the tests failed.` — Both sides are full sentences, so a comma goes before `but`.
5. `:` — `We need three things: a test, a migration, and a seeder.` — A colon introduces a list.
6. `Can you open a pull request for this?` — Add the article `a`, and end with a `?` because it's a request/question.
7. `I think we have so many bugs in this release.` — `bug` is countable, so `many bugs` (plural), not `much bug`; capital `I`.

8. **B** — **After the deploy, watch the error rate.** — A comma follows an introductory clause.
9. **Done. LGTM – ship it!** — Separate the statements; **LGTM** stays uppercase. (**Ship it!** may take a **!** since it's a strong go-ahead.)
- l0. **The API returns 500 when your token is expired.** — Capitalize **The** and **API**; **ur** → **your**; add the full stop.
- l1. **?** — **Did the migration run on staging?** — A direct question always ends with a question mark.
- l2. **I did research on grammar like you asked.** — Capital **I**, fix the spelling **reaseach** → **research** and **grammer** → **grammar**, and **u** → **you**.

Part of the Developer English course — see [00-index.md](#).

08 — Sentence structure: run-ons, fragments & comma splices

Source: [readme/research/english/08-sentence-structure-run-ons.md](#)

IN SHORT — KEY TAKEAWAYS

This document teaches you how to write clear, complete sentences — no more long chains of words with no full stops. If you often write one long line where three or four sentences should be, this is for you. Fixing this makes your messages, notes, and updates much easier for others to read and understand.

The main parts explained simply:

- **A complete sentence** — Every sentence needs three things: who or what (subject), what they do (verb), and a finished thought. “The build failed.” is complete. “the build failed i pushed a fix” is two sentences squashed into one — that is a problem.
- **Run-on sentence** — When you write two full thoughts with no break between them, that is called a run-on. It is like speaking without pausing for breath. Example: “the build failed i pushed a fix” — needs a full stop in the middle.
- **Comma splice** — When you put two full sentences together with only a comma, that is also wrong. A comma alone is too weak to join two sentences. “the tests pass, the deploy is green” needs a full stop, not just a comma.
- **Four correct ways to join or split sentences** — You have four safe choices: (1) use a full stop to make two separate sentences, (2) use a semicolon when the two ideas are very close, (3) use a comma plus a joining word like “and”, “but”, or “so”, (4) use a connecting word like “because”, “when”, or “if” to link the ideas.
- **Fragments** — A fragment is a sentence that is not finished. “Because the token expired.” sounds like it is waiting for more. It needs to attach to another sentence: “The request failed because the token expired.”
- **Daily writing practice** — The document shows how these rules apply to real work: writing commit messages, pull request descriptions, Slack standups, code comments, and bug reports. Short, clear sentences help people read faster and miss fewer things.
- **Drills and answer key** — The document ends with 12 practice exercises and full answers so you can test yourself and check your work right away.

What to do with this: When you finish writing a message or note, read it aloud. Every time you find two full ideas with no break between them, add a full stop or a joining word. Remember: a comma alone cannot join two sentences — always add “and”, “but”, or “so” after it.

What this fixes for you: your habit of writing one long sentence with no full stops, so your commits, PRs, and Slack messages read clearly the first time — no re-reading needed.

The rule in 30 seconds

- A **complete sentence** has three things: a **subject** (who/what), a **verb** (the action), and a **complete thought** (it can stand alone). Example: “The build failed.”
- **Run-on** = two complete sentences jammed together with no break: “the build failed i pushed a fix”. Wrong.
- **Comma splice** = two complete sentences joined by *only* a comma: “the build failed, i pushed a fix”. Still wrong — a comma is too weak to glue two sentences.
- You have **four legal ways** to join or separate them: a full stop `.`, a semicolon `;`, a comma + conjunction (`and` / `but` / `so`), or a subordinating word (`because` / `when` / `after` / `if`).

Why this trips you up

You chain everything into one breathless line and drop every full stop — exactly like your real message: *“can u do reaseach on grammer for me and teach me i already know but as of now i am making so much mistake.”* That is three or four sentences with no breaks, no capitals, and a couple of spelling and count errors riding along. In code you break logic into small functions; do the same with sentences — one idea per sentence, then stop.

See it / fix it

✗ What you tend to write	✓ Correct	Why
the build failed i pushed a fix	The build failed. I pushed a fix.	Run-on. Two thoughts → two sentences. Capital I , full stops added.
the tests pass, the deploy is green	The tests pass. The deploy is green.	Comma splice — a comma can't join two sentences. Use a full stop.
i refactored the service it is faster now	I refactored the service, and it is faster now.	Run-on. Comma + and joins two complete thoughts.
we hit a 429, we should add a retry	We hit a 429, so we should add a retry.	Comma splice. Add so to show the cause → effect.
cache was stale users saw old prices	Cache was stale, which made users see old prices.	Run-on. A subordinate clause (which...) attaches the second idea.
merged the PR (as a standalone Slack line)	I merged the PR.	Fragment — missing the subject. Add I .
because the token expired (alone)	The request failed because the token expired.	Fragment — a because clause can't stand alone; attach it to a full sentence.
i already know but as of now i am making so much mistake	I already know it, but right now I'm making so many mistakes.	Run-on + count error: mistake is countable → so many mistakes . Comma + but .
can u do reaseach on grammer	Can you do research on grammar?	Texting + spelling: u→you , reaseach→research , grammer→grammar ; it's a question → ? .
open pull request when ready it will trigger CI	Open a pull request when ready; it will trigger CI.	Missing article a ; run-on fixed with a semicolon (two related thoughts).

Patterns to remember

The four fixes — pick one when you have two complete sentences:

Fix	Looks like	Use when
Full stop	It failed. I fixed it.	The two ideas are independent. Safest default — use this most.
Semicolon	It failed; I fixed it.	The ideas are closely linked and you want them in one line.
Comma + conjunction	It failed, so I fixed it.	You want to show the relationship: and (add), but (contrast), so (result).
Subordinate clause	Because it failed, I fixed it.	One idea depends on the other: because / when / after / if / which .

Mnemonics:

- **“Two verbs, two subjects → needs a join or a stop.”** If you can split the line into two standalone sentences, you must use one of the four fixes between them.
- **“A comma is not glue.”** A comma alone never joins two sentences. It always needs a conjunction (`, and` / `, but` / `, so`).
- **Fragment test:** read the chunk out loud alone. If it leaves you waiting for more (“Because the token expired...”), it’s a fragment — attach it.

In your daily writing

- **Commit messages:** one line, one idea. `Fix stale cache so users see current prices` beats `fixed cache it was stale users saw old prices` .
- **PR descriptions:** write short sentences, each ending in a full stop. Reviewers scan; a wall of run-on text gets skimmed and bugs get missed.
- **Slack standups:** `Yesterday I finished the retry logic. Today I'm on the 429 handler. No blockers.` — three clean sentences, not one chain.
- **Code comments:** `// Token can expire mid-request, so we refresh before retrying.` — comma + `so` shows cause. Avoid `// token expired refresh and retry it works now` .
- **Bug reports:** separate *what happened* from *what you expected*. `The export returns an empty file. I expected the full order list.` Two sentences, instantly clear.

Drills

Fix or choose. Don’t peek at the answer key until you’ve tried each.

1. Fix: `the migration ran it dropped the wrong column we need a rollback`
2. Fix: `i opened the PR, the CI is still running`
3. Choose A or B: (A) `The cache was cold, so the first request was slow.` (B) `The cache was cold, the first request was slow.`
4. Fill the blank with the best join: `The API returned 500 ____ the database connection timed out.` (`.` / `;` / `because`)
5. Fix the texting + run-on: `can u review my pr i think its ready but tests are flaky`
6. Is this a complete sentence, a fragment, or a run-on? `Restarting the worker queue.`
7. Fix: `we shipped the feature flag off by default users wont see it yet`
8. Choose A or B: (A) `I rebased onto main but there were conflicts.` (B) `I rebased onto main, but there were conflicts.`
9. Fix the count + run-on: `i made so much mistake in this commit i will clean it up`
10. Fill the blank: `____ the token expired, the request failed.` (`Because` / `But` / `So`)
11. Fix: `deploy is green metrics look normal closing the incident`
12. Fix the article + comma splice: `open pull request, it triggers the build`

Answer key

1. **The migration ran. It dropped the wrong column, so we need a rollback.** — Split the first run-on with a full stop; join the cause/result with **, so**.
2. **I opened the PR. The CI is still running.** — Comma splice fixed with a full stop. Also capitalize **I**.
3. **A. , so** correctly joins two complete sentences. B is a comma splice (comma alone can't join them).
4. **because** (or a full stop): **The API returned 500 because the database connection timed out.** — The second clause explains the first, so **because** reads best; **.** also works.
5. **Can you review my PR? I think it's ready, but the tests are flaky.** — **u→you**, **its→it's** (it is), question mark, and **, but** joins the contrast.
6. **Fragment** — no subject or finished thought. Fix: **I'm restarting the worker queue.**
7. **We shipped the feature. The flag is off by default, so users won't see it yet.** — Break the run-on; **, so** shows the result. Note **won't** with the apostrophe.
8. **B.** When **but** joins two complete sentences ("I rebased onto main" + "there were conflicts"), put a comma before it. A is missing the comma.
9. **I made so many mistakes in this commit. I'll clean it up.** — **mistake** is countable → **so many mistakes** with the plural **-s**; split the run-on with a full stop.
10. **Because** — **Because the token expired, the request failed.** The clause sets up the cause; note the comma after the opening subordinate clause.
11. **Deploy is green. Metrics look normal. I'm closing the incident.** — Three separate thoughts → three sentences (the last needs a subject, **I'm**).
12. **Open a pull request; it triggers the build.** — Add the article **a**; replace the comma splice with a semicolon (closely linked ideas) or a full stop.

Part of the Developer English course — see [00-index.md](#).

09 — Commonly confused words

Source: [readme/research/english/09-commonly-confused-words.md](#)

IN SHORT — KEY TAKEAWAYS

This document is about small word mix-ups that are very easy to make in English — things like writing “its” when you mean “it’s”, or “login” when you should write “log in”. These are not big grammar rules — they are tiny word choices. But getting them wrong makes your writing look careless, even when your actual work is good.

The main parts explained simply:

- **The apostrophe test (it’s / its, you’re / your, they’re / their, who’s / whose)** — When you see a word with an apostrophe like “it’s”, it is always short for two words (“it is” or “it has”). If you cannot expand it to two words, do not use an apostrophe. Example: “its config” means the config belongs to it — no apostrophe needed.
- **lose vs loose** — “Lose” (one “o”) is the action of misplacing something: “don’t lose the data.” “Loose” means not tight: “a loose cable.” These are spelled differently and mean different things.
- **then vs than** — “Than” is for comparisons: “faster than before.” “Then” is for time or order: “first build, then deploy.”
- **affect vs effect** — “Affect” is the action (the verb): “this will affect the cache.” “Effect” is the result (the noun): “what is the effect on the cache?” A shortcut: Affect = Action, Effect = End result.
- **One word as a thing, two words as an action (login / log in, setup / set up, backup / back up)** — When you name a thing, it is one word: “the login screen”, “the setup guide”. When you describe doing the action, it is two words: “please log in”, “set up the server.”
- **fewer vs less** — If you can count the items (3 errors, 5 tickets), use “fewer.” If you cannot count it (less memory, less time), use “less.”
- **accept vs except** — “Accept” means to agree or receive something. “Except” means to leave something out: “all tests pass except one.”
- **e.g. vs i.e.** — “e.g.” means “for example.” “i.e.” means “that is” or “in other words.” Both need a comma after them.
- **Practice drills and answer key** — The document ends with 14 short exercises and full answers so you can check yourself right away.

What to do with this: Before you send a commit message or Slack message, do one quick check — scan for “its/it’s”, “lose/loose”, “then/than”, and “login/log in”. Those four catch most mistakes. If a word has an apostrophe, say it out loud as two words — if it sounds wrong, remove the apostrophe.

What this fixes for you: the small word mix-ups that make a clean PR or commit look careless — its/it’s, your/you’re, login vs log in, and friends. Get these right and your writing instantly reads as more senior.

The rule in 30 seconds

- If you can expand it to two words (**it is**, **you are**, **they are**), use the apostrophe version (**it's**, **you're**, **they're**). If not, use the plain one (**its**, **your**, **their**).
- **then** = time/sequence; **than** = comparison.
- **affect** = the verb (to change something); **effect** = the noun (the result).
- Many of these are one word as a *noun* (**login**, **setup**, **backup**) but two words as a *verb* (**log in**, **set up**, **back up**).

Why this trips you up

You write fast, all day, in commits and Slack — and texting habits leak in (“u”, “ur”, no capitals), so the apostrophe and the missing letter slip past. You also tend to drop the small grammar signals (the plural **-s**, the article **a/the**), and these confusable pairs hide in exactly that same blind spot. None of this means you don't know the rule — it means you're moving fast and need a quick self-check.

See it / fix it

✗ What you tend to write	✓ Correct	Why
fix: button looses focus on close	fix: button loses focus on close	lose = a verb (to misplace). loose = not tight. One o .
your right, the migration was missing	you're right, the migration was missing	you're = you are.
the API returns it's status code	the API returns its status code	its = belonging to it. No apostrophe (like <i>his, hers</i>).
this build is slower then the last one	this build is slower than the last one	Comparison → than .
how does this affect the cache? (meaning the result)	what's the effect on the cache?	Result/outcome → effect (noun).
please login to the dashboard and retry	please log in to the dashboard and retry	Here it's a verb → two words: log in .
i updated the setup steps, see PR	I updated the setup steps. See the PR.	Noun setup is fine; but capitalize I , end the sentence, add the .
there going to revert there changes	they're going to revert their changes	they're = they are; their = belonging to them.
accept for the timeout, all tests pass	except for the timeout, all tests pass	except = leaving out. accept = to agree/receive.
we had less errors after the fix	we had fewer errors after the fix	Countable (errors) → fewer . Uncountable (downtime) → less .
who's branch is this?	whose branch is this?	Ownership → whose . who's = who is.
use JSON, e.g. the response body (meaning "that is")	use JSON, i.e. the response body	i.e. = that is (clarify). e.g. = for example.

Patterns to remember

The apostrophe = “two words” test

Word	Expands to?	Use when
it's	it is / it has	"it's failing" = it is failing
its	(no expansion)	"its config" = the config belonging to it
you're	you are	"you're blocked"
your	(no expansion)	"your branch"
they're	they are	"they're merging"
their	(no expansion)	"their service"
there	(no expansion)	"deploy it there" / "there is a bug"
who's	who is / who has	"who's on call?"
whose	(no expansion)	"whose PR broke main?"

One word (noun) vs two words (verb) — the verb has a space:

Noun (one word)	Verb (two words)
a login screen	please log in
the setup is done	set up the env
run a backup	back up the DB

Trick: if you can put a word between the two halves ("set the env **up**", "log quickly **in**"), it's a verb → two words.

then vs than — **a** for compare (than / compare), **e** for sequence (then / first then next).

affect vs effect — **Affect** = **A**ction (verb). **Effect** = **E**nd result (noun).

fewer vs less — if you can count them with an **-s** plural (3 errors, 5 retries) → **fewer**. If you can't (less traffic, less memory) → **less**.

to / too / two — **too** = also or excessive ("too slow"). **two** = the number 2. **to** = everything else.

into vs in to — **into** = movement/result ("merge it into main"). **in to** = the words just land next to each other ("log **in to** the box" = log in + to the box).

accept vs except — **accept** = take it; **except** = **e**xclude it.

In your daily writing

- **Commit messages:** the big four to scan for are **its/it's**, **lose/loose**, **then/than**, **login/log in**. Example: `fix: prevent session from being lost on log in` (verb → two words).
- **PR titles:** comparisons sneak in — "smaller bundle **than** before", not "then".
- **Slack standups:** kill the texting leak. Write "your PR is ready", not "ur PR is ready"; "they're reviewing", not "there reviewing". Capitalize **I** and the first word.

- **Code comments & docs:** use **e.g.** for examples and **i.e.** to restate one specific thing — and put a comma after both: `// retry on transient errors, e.g., 502 and 503`.
- Quick self-check before you hit send: any apostrophe word? Expand it in your head. Any “login/setup”? Ask “is it a thing or an action?”

Drills

Fix or choose. Don't peek at the key.

1. Fix: `your going to want to rebase before you push`
 2. Choose: The new query is faster (then / than) the old one.
 3. Fix: `the cache keeps it's value after restart`
 4. Choose: Please (login / log in) and run the seeder.
 5. Fix: `we have less open tickets then last sprint`
 6. Choose: This change won't (affect / effect) the public API.
 7. Fix: `accept for one flaky test, the suite is green`
 8. Fill the blank: `___ branch is this, and ___ reviewing it?` (whose / who's)
 9. Fix (rewrite the learner's own line): `can u do reaseach on grammer for me and teach me i already know but as of now i am making so much mistake`
 10. Choose: Merge the hotfix (into / in to) main.
 11. Fix: `there service is down so there going to fail over`
 12. Choose: Use a real client, (e.g. / i.e.) curl or Postman.
 13. Fix: `the build looses the env vars during setup` (one error is a word-choice, find it)
 14. Choose: The retry count is (to / too / two) high.
-

Answer key

1. **You're going to want to rebase before you push.** — “you're” = you are; capitalize the first word; end with a full stop.
2. **than** — it's a comparison.
3. **The cache keeps its value after restart.** — “its” = belonging to the cache, no apostrophe.
4. **log in** — it's a verb here, so two words.
5. **We have fewer open tickets than last sprint.** — tickets are countable → “fewer”; comparison → “than”.
6. **affect** — it's the verb (to change something).
7. **Except for one flaky test, the suite is green.** — “except” = leaving out; capitalize the first word.
8. **Whose** branch is this, and **who's** reviewing it? — “whose” = ownership; “who's” = who is.

9. **Can you do research on grammar for me and teach me? I already know it, but right now I'm making so many mistakes.** — fixes: “you” not “u”, spelling “research”/“grammar”, capital “I”, a question mark, “many mistakes” (countable plural), and clearer wording.
- L0. **into** — one word for movement/result (merging X into Y).
- L1. **Their service is down, so they're going to fail over.** — “their” = belonging to them; “they're” = they are; comma before “so”.
- L2. **e.g.** — you're giving examples, not restating one exact thing.
- L3. **The build loses the env vars during setup.** — “loses” (verb, one ); “setup” as a noun here is correct.
- L4. **too** — meaning excessive.
-

Part of the Developer English course — see [00-index.md](#).

10 — Professional dev writing: commits, PRs, Slack, comments

Source: [readme/research/english/10-professional-dev-writing.md](#)

IN SHORT — KEY TAKEAWAYS

This document teaches you how to write like a senior developer — not by coding better, but by writing better. Every day you write commit messages, pull request descriptions, code review comments, Slack updates, and code comments. Small writing mistakes in these places make solid work look unpolished. Fix them once and your writing reads as professional immediately.

The main parts explained simply:

- **Commit messages** — The one-line title of every save you make to the codebase. Always start with a command word (“Fix”, “Add”, “Update”), capitalize the first letter, and do not put a full stop at the end. Wrong: `fixed the bug`. Right: `Fix cart total when quantity is zero`
- **Pull request (PR) descriptions** — The summary you write when you ask a teammate to review your code. The title follows the same commit rules. The body answers three questions in full sentences: What did you change? Why? How can someone test it?
- **Code review comments** — Notes you leave on someone else’s code. Never just say “wrong” or “no.” Name the problem and suggest a fix. A question (“Could we guard this?”) feels helpful; a command (“fix this”) feels harsh.
- **Slack and standup messages** — Short daily updates about your work. Write full sentences with capital letters and full stops — not texting shorthand. Three sentences: yesterday, today, blockers.
- **Code comments** — Notes written inside the code itself. Always present tense, full sentence, capitalized. “Returns the cached price.” — not “return cached price” or “this is returning...”
- **Much vs many** — A grammar rule that trips people up often. If you can count it (bugs, mistakes, tests), use **many**. If you cannot count it (time, work, research), use **much**.
- **Articles (a / an / the)** — Small words that English almost always needs before nouns. “Open **a** pull request” not “open pull request.” When in doubt with a single countable thing, it needs *a*, *an*, or *the*.
- **Spelling watchlist** — Words commonly misspelled: *grammar* (not grammer), *research* (not reaseach), *separate*, *definitely*, *environment*.

What to do with this: Start with commit messages — use the formula `Fix/Add/Update + what + where`, no trailing period. Then move to Slack: write three full sentences for every standup, no `u / ur / pls`. These two habits alone make the biggest visible difference.

What this fixes for you: this is the capstone. It takes every grammar rule from the earlier lessons and applies it to the things you actually type all day — commit messages, pull requests, code review comments, Slack standups, code comments, and emails. Get these right and your work reads as senior, no matter your accent or first language.

The rule in 30 seconds

- **Commits:** imperative mood, capitalized first word, no period at the end of the subject line. *“Fix null guard on checkout”* — not *“fixed the null guard.”* or *“fixes null”*.
- **PRs, comments, Slack, docs:** write **complete sentences** — capital letter at the start, full stop at the end, no texting shortcuts (`u` , `ur` , `pls`).
- **Be specific, not blunt:** in reviews, say what’s wrong *and* suggest the fix. *“This throws when input is null”* beats *“this wrong.”*
- **Code comments and docstrings:** **present tense**, full sentence. *“Returns the cached price.”* — not *“return cached price”* or *“this is returning...”*

Why this trips you up

You’re a fast typist under deadline, so texting habits leak into work writing — lowercase `i` , `u` for *you*, no full stops, and long run-on sentences that chain clauses with no commas. You also tend to drop the small words: articles (`open pull request` instead of *open **a** pull request*) and plural `-s` (*“so much mistake”* instead of *“so **many** mistakes”*). None of these break the build, but together they make solid engineering work look unpolished. The fixes are mechanical — once you see the pattern, you stop making the mistake.

See it / fix it

✗ What you tend to write	✓ Correct	Why
<code>fixed the bug in cart</code>	<code>Fix cart total when quantity is zero</code>	Commit subject: imperative mood, capitalized, specific, no period.
<code>Update readme.</code>	<code>Update README with setup steps</code>	Drop the trailing period in the subject; say <i>what</i> changed.
<code>i opened pull request for review</code>	<code>I opened a pull request for review.</code>	Capital <code>I</code> ; add the article <code>a</code> ; end with a full stop.
<code>this wrong</code> (review comment)	<code>This looks like it will throw when input is null – should we guard it?</code>	Complete sentence, names the problem, suggests a fix politely.
<code>u need to handle ur error here</code>	<code>You need to handle your error here.</code>	No texting (<code>u</code> / <code>ur</code>); capital first word; full stop.
<code>i did reaseach on the grammer issue</code>	<code>I did research on the grammar issue.</code>	Spelling: <i>research</i> , <i>grammar</i> ; capital <code>I</code> .
<code>there is so much mistake in this PR</code>	<code>There are so many mistakes in this PR.</code>	Countable noun: <i>many mistakes</i> ; verb agrees (<i>are</i>).
<code>// return cached price</code>	<code>// Returns the cached price.</code>	Code comment: present tense, full sentence, capitalized.
<code>standup: did login fix, doing cart, blocked on api</code>	<code>Yesterday I finished the login fix. Today I'm working on the cart. I'm blocked on the API.</code>	Full sentences with punctuation read as a clear update.
<code>plan grammar docs for as per that</code>	<code>I'll plan the grammar docs based on that.</code>	Fix word order and the awkward “ <i>for as per that</i> ”.
<code>pls review when u get time thanks</code>	<code>Please review when you get a chance. Thanks!</code>	No <code>pls</code> / <code>u</code> ; add the article; clean sign-off.
<code>error: cannot process</code> (error message)	<code>Could not process the file. Please try again.</code>	User-facing text: complete sentence, plain-language recovery.

Patterns to remember

The commit subject formula: `<Verb in imperative> <what> [<where/condition>]`

- Test: the subject should finish the sentence “*If applied, this commit will...*”. → “*...Fix cart total when quantity is zero.*” ✓
- Imperative verbs to start with: **Add, Fix, Remove, Update, Refactor, Rename, Bump, Document, Revert.**

- ❌ Added , Fixes , Fixing , fix → ✅ Add , Fix (capitalized, base form).
- No period at the end of the **subject**. Periods are fine in the **body**.

Commit body (after a blank line): explain *why*, in full sentences. The diff already shows *what* changed; the body should answer *why* it needed changing.

Much vs many (your recurring one):

Uncountable → much	Countable → many
much code, much work, much progress	many bugs, many mistakes, many tests
much feedback, much research	many commits, many endpoints, many files

If you can put a number in front of it (3 bugs), it's countable → use **many** and add the **-s**.

Articles in one line: use **a/an** for one unspecified thing (open **a** PR), **the** for a specific known thing (merge **the** PR you opened). When in doubt with a singular countable noun, it almost always needs *a*, *an*, or *the* in front.

Spelling watchlist: grammar (not *grammer*), research (not *reaseach*), separate, definitely, occurred, received, environment, length.

In your daily writing

- **Commits:** one imperative line under ~50 characters, capitalized, no period. If the *why* isn't obvious, add a blank line and a body in full sentences.
- **PRs:** the **title** is a commit-style summary. The **description** answers three questions in complete sentences: **What** does this change? **Why** is it needed? **How** did you do it (and how can a reviewer test it)?
- **Code review comments:** never just "*wrong*" or "*no*." State the issue, then propose a path: "*This will fail for empty arrays — can we default to `[]` ?*" Questions feel collaborative; commands feel harsh.
- **Slack / standup:** treat it like a short report, not a text to a friend. Capital letters, full stops, no **u / ur**. Three sentences: yesterday, today, blockers.
- **Code comments & docstrings:** present tense, full sentence, describe behavior. "*Validates the upload and returns the relative path.*"
- **Emails:** greeting (*Hi Sarah,*) → one clear ask → sign-off (*Thanks, Pravin*). Don't bury the request in the middle.

Drills

1. Rewrite as a proper commit subject: fixed the issue where login was failing sometimes.
2. Rewrite as a proper commit subject: adding new endpoint for invoices
3. Fix this sentence: i think there is so much bug in this function we should add test.
4. Fix the review comment so it's polite and specific: this wrong, change it

5. Fix the Slack message: `done with cart, starting checkout now, blocked on payment gateway`
 6. Choose A or B: We should (A) *open pull request* / (B) *open a pull request* before merging.
 7. Fix spelling and capitalization: `i did reaseach on the grammer rules and made a doc.`
 8. Rewrite as a present-tense code comment: `// this is checking if the user is logged in`
 9. Fill the blank with *much* or *many*: There were too _____ failing tests to release today.
 10. Fill the blank with *much* or *many*: We don't have _____ time before the deadline.
 11. Polish this email line into a greeting + clear ask + sign-off: `pls deploy the branch when u can thanks`
 12. Fix the run-on into clean sentences: `i opened the PR it has the cart fix and the null guard can u review it today.`
-

Answer key

1. **Fix intermittent login failure** — imperative verb, capitalized, specific, no trailing period.
 2. **Add invoices endpoint** — imperative *Add* (not *adding*); concise; no period.
 3. **I think there are so many bugs in this function. We should add tests.** — capital **I** ; countable *many bugs*; verb *are*; full stop splits the run-on; plural *tests*.
 4. **This looks wrong to me – I think it breaks when the list is empty. Could we guard it?** — names the problem, suggests a fix, stays polite with a question.
 5. **Yesterday I finished the cart. Today I'm starting checkout. I'm blocked on the payment gateway.** — three full sentences, capitalized, punctuated; adds *the*.
 6. **B** — *open a pull request*; a singular countable noun needs an article.
 7. **I did research on the grammar rules and made a doc.** — spelling *research* and *grammar*; capital **I** ; article *the*.
 8. **// Checks whether the user is logged in.** — present tense, full sentence, capitalized; drop the wordy “*this is checking*.”
 9. **many** — *tests* are countable, so *too many*.
 10. **much** — *time* is uncountable, so *not much time*.
 11. **Hi [name], could you deploy the branch when you get a chance? Thanks, Pravin** — greeting, clear ask as a polite question, sign-off; no `pls / u`.
 12. **I opened the PR. It has the cart fix and the null guard. Could you review it today?** — split into three sentences; capital **I** ; `you` not `u` ; the last clause becomes a polite question.
-

11 — Spelling & typos developers get wrong

Source: [readme/research/english/11-spelling-typos.md](#)

IN SHORT — KEY TAKEAWAYS

This document is about the small spelling mistakes that developers make every day — in commit messages, pull request descriptions, Slack messages, and code comments. It matters because a misspelled word in a professional message makes careful work look careless, and most of these mistakes come from speed and habit, not from not knowing the word.

The main parts explained simply:

- **The “always misspelled” short list** — A set of about 12 words that almost every developer gets wrong at least sometimes: words like “separate”, “definitely”, “recommend”, “occurred”, and “environment”. The document gives a memory trick for each one so you can fix the habit once and for all.
- **The i-before-e rule** — A simple spelling pattern: write “i” before “e” (believe, achieve, field), but flip it to “e” before “i” after the letter “c” (receive, deceive). Covers the words you type most.
- **The doubling rule** — When you add “-ing” or “-ed” to a short word where the stress lands on the last sound, you double the last letter (run → running, commit → committed). When the stress does NOT land on the last sound, you do not double it (open → opening). One quick pattern to check.
- **Words that sound alike but mean different things** — Pairs that trip people up: “which” vs “witch”, “parameter” (a function input) vs “perimeter” (a boundary), and “a lot” (two words, always) vs the non-word “alot”. Swapping these in a PR looks like a knowledge gap, not just a typo.
- **Texting habits leaking into work writing** — Shortcuts like “u”, “ur”, “plz”, and “thx” are fine in personal chats but read as unprofessional in team Slack, PR descriptions, and tickets. The fix is just writing the full word.
- **Where these mistakes actually appear** — Commit messages, PR titles, Slack standups, code comments, and bug reports. The document shows real before-and-after examples for each place so the habit change feels concrete.
- **Practice drills with answers** — Twelve short exercises that reproduce the exact mistakes from the “See it / fix it” table. You can use these to check whether a correction has actually stuck.

What to do with this: Pin the “always misspelled” word list somewhere visible (a sticky note or phone note). Turn on spell-check in your code editor and in whatever tool you use to write PR descriptions. Then before sending any Slack message to the team, spend two seconds reading it back — spell out “you”, “your”, “please”, and “thanks” in full.

What this fixes for you: the small misspellings that slip into commits, PRs, and Slack — “grammer”, “reaseach”, dropped or doubled letters — plus the texting habits (“u”, “ur”, “plz”) that make professional writing look rushed.

The rule in 30 seconds

- A few words get misspelled by almost everyone: **separate, definitely, recommend, occurred, necessary, receive, environment, parameter, maintenance**. Learn these as fixed shapes — don't sound them out.
- **i before e, except after c** (recieve → **receive**, beleive → **believe**).
- Doubling rule: short stressed words double the final consonant before **-ing/-ed** (run → running, occur → occurred) — but **open** → **opening** (not stressed on the last part).
- Spell-check is not cheating. Turn it on in your editor and your PR/Slack box; let the machine catch what your eyes skim past.

Why this trips you up

You write fast all day — commits, standups, review comments — and the same slips repeat: “grammer” for **grammar**, “reaseach” for **research**, plus texting shortcuts (“u”, “ur”) leaking from your phone into work. Typing speed is good; the fix is a 2-second re-read and spell-check, not slowing down. None of these mean you don't know the word — they're muscle-memory typos, and muscle memory is easy to retrain.

See it / fix it

✗ What you tend to write	✓ Correct	Why
can u do reaseach on grammer for me	can you do research on grammar for me	“grammar” ends in -ar ; “research” is re- search ; spell out you
ur branch is out of date	your branch is out of date	“ur” is texting; write your
plz review thx	please review, thanks	“plz/thx” are chat shortcuts, not PR English
this seperate module	this separate module	there’s a rat in sep- a-ra-te
definatly a bug	definitely a bug	it’s finite inside: de- finite-ly
recieve the webhook payload	receive the webhook payload	i before e except after c
the error occured on retry	the error occurred on retry	double the r : oc- curr -ed
recomend merging	recommend merging	one c , two m : re- comm -end
set the enviroment variable	set the environment variable	don’t drop the n : environ-ment
missing paramter in the call	missing parameter in the call	para- meter , like a measuring meter
this is depricated, dont use it	this is deprecated , don’t use it	depre -cated (not “depri”); add the apostrophe in don’t
async call is asynchronous	the call is asynchronous	a- syn-chro -nous (has “chron”, like chronology)
db dependancy needs updating	db dependency needs updating	ends in -ency : depend- ency

Patterns to remember

The dev-writing “always misspelled” wall. Pin these where you can see them:

Word	Hook to remember it
separate	“there’s a rat in separate”
definitely	hidden word finite
recommend	one c, two m
occurred / occurring	double c, double r
necessary	one c, two s — “one collar, two sleeves”
environment	keep the n before -ment
parameter	a “meter” measures a parameter
length / strength	-ength, not -enght
succeed	two c, two e (but “ success ”, “ successful ”)
beginning	double the n before -ing
until	one l at the end (not “untill”)
maintenance	-enance , not “-ainance”

i before e:

- Yes: believe, achieve, retrieve, **field**.
- After c, flip: receive, deceive, **ceiling**.
- (Real-world exceptions exist — “weird”, “seize” — but the rule covers the words you type most.)

Doubling before -ing / -ed (one short stressed syllable → double):

- run → **running**, set → **setting**, commit → **committed**, refer → **referred**.
- But NOT when stress isn’t on that syllable: **open** → **opening**, **offer** → **offered**, **enter** → **entered**.

Words that sound alike — don’t swap them:

Word	Means	Dev example
which	the relational one	“the branch which broke CI”
witch	Halloween	(never in a PR)
parameter	a function input	“pass the timeout parameter ”
perimeter	a boundary length	(geometry, not your API)
a lot	two words, “many”	“fixed a lot of typos”
alot	not a word	X

In your daily writing

- **Commits:** “fix: handle null **parameter** in webhook **receive** handler” — re-read the subject line once; a typo there is permanent in history.
- **PR titles/descriptions:** turn on your editor’s spell-check for Markdown. “**Separate** the **environment** config” reads as careful; “Seperate the enviroment config” reads as rushed.
- **Slack standups:** write **you, your, please, thanks** in full. “plz check ur PR thx” is fine with a friend, not in a team channel.
- **Code comments:** `// receive the response, then retry if it occurred to fail` — comments live in the codebase forever; spell them like docs.
- **Tickets/bug reports:** “**Definitely** reproducible — the error **occurred** after the **asynchronous** call.” Clear spelling makes the report trustworthy.

Drills

Fix the spelling/word issue in each (some have more than one). Don’t peek at the key.

1. Fix this Slack message: “can u do reaseach on grammer for me”
2. Fix: “this needs a seperate enviroment for testing”
3. Fill the blank: “I before E except after ” — **and spell the verb for “to get a payload”**: re_.
4. Choose A or B: The error (A) occured (B) occurred during the async retry.
5. Fix: “I definately recomend we merge this”
6. Fill the blank: commit → commi_____ (add the -ed form).
7. Choose A or B: We need to pass the (A) parameter (B) perimeter to the function.
8. Fix the texting style: “plz review ur branch thx”
9. Fill the blank: open → _____ing; run → _____ing (mind the doubling rule).
10. Fix: “this api is depricated and the dependancy is old”
11. Choose A or B: The branch (A) witch (B) which broke CI.
12. Fix all the typos: “the maintainance window will continue untill the migration succeds”

Answer key

1. “**Can you do research on grammar for me?**” — spell out **you**; **research** (re-search); **grammar** ends in -ar; capitalize the first word and add a question mark.
2. “**this needs a separate environment for testing**” — **separate** (“a rat”); **environment** keeps the **n**.
3. **except after C**; **receive** — i before e, but flip after c.
4. **B — occurred** — double the c and r before -ed.
5. “**I definitely recommend we merge this**” — **definitely** (hidden “finite”); **recommend** (one c, two m).
6. **committed** — short stressed syllable doubles the **t** before -ed.

7. **A — parameter** — a function input; “perimeter” is a boundary length.
 8. **“Please review your branch. Thanks!”** — full words, capital first letter, full stop.
 9. **opening; running** — “open” isn’t stressed on the last part, so no doubling; “run” is, so double the **n**.
 10. **“this API is deprecated and the dependency is old”** — **deprecated** (not “depri-”); **dependency** ends in -ency; “API” is uppercase.
 11. **B — which** — relational word; “witch” is Halloween.
 12. **“the maintenance window will continue until the migration succeeds”** — **maintenance** (-enance); **until** (one l); **succeeds** (two c, two e, plus the -s).
-

Part of the Developer English course — see [00-index.md](#).

12 — Master mixed review

Source: [readme/research/english/12-master-mixed-review.md](#)

IN SHORT — KEY TAKEAWAYS

This is the final practice session of the Developer English course. It mixes every grammar rule from all previous lessons into one big exercise so you can check what you have learned and what you still need to fix. It matters because writing clean, professional English in commits, messages, and reports makes your work easier for your team to understand and trust.

The main parts explained simply:

- **Quick reminder table** — A one-page cheat sheet showing every rule covered in the course and the single most important thing to remember for each one. Keep it beside you while you do the exercises.
- **Part A — Fix the sentence** — Ten real-looking sentences that developers actually write, each with one or more mistakes. You rewrite them correctly. This trains your fingers, not just your brain.
- **Part B — Fill the blank** — Ten sentences where one word is missing. You pick the right word from two or three choices. Focuses on the trickiest pairs: much/many, its/it's, your/you're, then/than.
- **Part C — Choose A or B** — Four pairs of sentences where only one is correct. Tests possessives, subject-verb agreement, tenses, and spelling.
- **Part D — Clean up the messy dev message** — Four badly written real-world messages (a Slack standup, a commit message, a PR description, and the very first sentence that started this course). You rewrite each one so a teammate can read it without effort.
- **Answer key with explanations** — Every answer is given with a one-line reason so you understand *why*, not just *what* is correct.

What to do with this:

Work through all 28 exercises with a pen or in a text file before you look at the answers. After checking, note which sections you got wrong most, then go back and re-read that lesson. The three habits that will improve your writing the fastest: always end a sentence with a full stop, always use a capital **I**, and before writing “much” or “many” ask yourself “can I count it?” — if yes, use **many** and add an **-s** to the noun.

What this fixes for you: this is the capstone. One big practice set that mixes every topic so far — articles, much/many, agreement, tenses, possessives, prepositions, punctuation, run-ons, confused words, and spelling — so the rules stick instead of fading after each separate lesson.

This doc is mostly **drills**. There's a short warm-up, then 28 mixed exercises (including a few “clean up this messy dev message” tasks), then a full answer key with one-line explanations. Work through it with a pen or in a scratch file, then check yourself.

How to use this

- Don't peek at the answer key until you've tried the whole set.
- Write your fix out in full — don't just think “yeah, I know that one.” The point is to train your fingers, because you write commits and Slack messages on autopilot.
- After checking, note which topics you missed most. Re-read that lesson (see [00-index.md](#)).

A quick reminder of what's being tested

Topic	The one thing to remember
Articles	Don't drop a / an / the : “open a pull request”, not “open pull request”.
Much / many	many + countable plural (“many bugs”); much + uncountable (“much progress”).
Agreement	Singular subject → singular verb: “the test passes ”, “the tests pass ”.
Tenses	“I fixed it” (done), “I have fixed it” (done, still relevant), “I'm fixing it” (now).
Possessives	its = belonging to it; it's = it is. your vs you're .
Prepositions	“merge into main”, “depends on the API”, “blocked by the migration”.
Punctuation	End sentences with a full stop. Use commas to break up clauses.
Capitalization	Always capital I . Capitalize the first word of every sentence.
Run-ons	Two complete thoughts need a full stop, a semicolon, or a comma + and/but.
Confused words	their/there/they're, then/than, affect/effect, lose/loose.
Spelling	grammar, research, separate, receive, occurred, definitely.
Texting leak	No “u”, “ur”, “thx” in professional writing. Write it out.

Drills

Part A — Fix the sentence

Each line has one or more errors. Rewrite it correctly.

1. i pushed the fix to ur branch can u review it
2. we are making so much mistake in the checkout flow
3. the migration are still running, it has not finished yet
4. please open pull request against main branch
5. i did a lot of reaseach on grammer before writing this doc
6. the API return 500 when the cart is empty we should add a guard

7. your right, the cache key was wrong its fixed now
8. there is to many open tickets in the backlog this sprint
9. the deploy effected all tenants because the config was shared
10. i think this approach is more better then the old one

Part B — Fill the blank

Choose the right word and write the full sentence.

11. The pricing service depends _____ the tenant context being initialized first. (*on / from / to*)
12. We merged the hotfix _____ main and tagged a release. (*into / in / to*)
13. There _____ three failing tests in the payment suite right now. (*is / are*)
14. _____ the queue worker still running, or did it crash? (*Is / Are*)
15. I've reviewed the PR; _____ ready to merge once CI passes. (*its / it's*)
16. Please update _____ local `.env` before running the seeder. (*your / you're*)
17. We didn't have _____ time to write tests for the edge cases. (*much / many*)
18. There were _____ duplicate rows after the bad import. (*much / many*)
19. The bug only appears _____ the user has more than one address. (*then / when*)
20. This fix is faster _____ the previous one by about 40 ms. (*then / than*)

Part C — Choose A or B

21. (A) "The endpoint returns it's payload as JSON." (B) "The endpoint returns its payload as JSON."
22. (A) "Each tenant have their own subdomain." (B) "Each tenant has its own subdomain."
23. (A) "I have already deployed it yesterday." (B) "I deployed it yesterday."
24. (A) "We need to separate the read and write paths." (B) "We need to sepearate the read and write paths."

Part D — Clean up the messy dev message

Rewrite each so a teammate could read it without effort. Fix spelling, capitals, full stops, run-ons, articles, much/many, and the texting style.

25. **A run-on Slack standup:** yesterday i worked on the cart bug today i am doing the checkout flow i found so much issue in the validation also the API is slow we should look at it tmrw can u help
26. **A sloppy commit message:** fixed stuff in pricing also some other thing, broke the test but its fine i think will fix latter
27. **A confusing PR description:** this PR change the order service so it dont crash when address is missing i also did some refactor and remove dead code there is still much bug but this is good for now plz review

28. **The learner's own line (the one that started this course):** can u do reaseach on grammer for me and teach me i already know but as of now i am making so much mistake
-

Answer key

Part A

1. **"I pushed the fix to your branch. Can you review it?"** — Capital **I**, "your" not "ur", "you" not "u", split into two sentences, and it's a question so it ends with **?**.
2. **"We are making so many mistakes in the checkout flow."** — "mistake" is countable, so **many** + plural **mistakes**. This is your signature slip.
3. **"The migration is still running; it hasn't finished yet."** — "migration" is singular → **is**. Two thoughts joined; use a semicolon (or a full stop) instead of a comma. (A comma alone between two complete sentences is a comma splice.)
4. **"Please open a pull request against the main branch."** — Add **a** before "pull request" and **the** before "main branch". You routinely drop these articles.
5. **"I did a lot of research on grammar before writing this doc."** — Capital **I**, "research" (not "reaseach"), "grammar" (not "grammer"). Memorize both spellings — they show up constantly in your work.
6. **"The API returns a 500 when the cart is empty. We should add a guard."** — "The API" is singular → **returns**; add **a** before "500"; split the run-on into two sentences.
7. **"You're right. The cache key was wrong; it's fixed now."** — "You're" = "you are"; "it's" = "it is" (here it means "it is fixed"). New sentence after "right".
8. **"There are too many open tickets in the backlog this sprint."** — "tickets" is plural → **are**; **too** (excessive), not "to"; **many** with countable tickets.
9. **"The deploy affected all tenants because the config was shared."** — **affected** (verb) not "effected"; "effect" is usually the noun. Memorize: *affect* = *action*, *effect* = *result*.
10. **"I think this approach is better than the old one."** — Capital **I**; "more better" is doubled — just **better**; **than** for comparison, not "then".

Part B

11. **on** — "depends **on** something." Fixed preposition.
12. **into** — you merge changes **into** a branch (movement toward a target).
13. **are** — "three failing tests" is plural.
14. **Is** — "the queue worker" is singular; also capital at the start of the sentence.
15. **it's** — "it's ready" = "it is ready."
16. **your** — possessive: the `.env` belonging to you. ("you're" = "you are".)

17. **much** — “time” is uncountable.

18. **many** — “rows” is countable plural.

19. **when** — the bug appears **when** a condition is met. (“then” = next/after.)

20. **than** — comparison uses **than**.

Part C

21. **B** — “its payload” (belonging to it). “it’s” would mean “it is payload,” which is nonsense.

22. **B** — “Each tenant” is singular → **has + its**. (“Each” is always singular, even though it feels like many.)

23. **B** — “yesterday” is a finished time, so use simple past **deployed**, not present perfect “have deployed.”
Don’t mix “already...yesterday.”

24. **A** — **separate** is the correct spelling. Mnemonic: there’s **a rat** in separate.

Part D

25. **Run-on Slack standup — cleaned up:**

Yesterday I worked on the cart bug. Today I’m on the checkout flow. I found many issues in the validation. The API is also slow — we should look at it tomorrow. Can you help?

What changed: capital **I** throughout; full stops between each thought (no more run-on); **many issues** (countable) not “so much issue”; “tomorrow” not “tmrw”; “you” not “u”; the question gets a **?**.

26. **Sloppy commit message — cleaned up:**

Fix pricing rounding bug; also adjust the tier lookup. One test is failing — will fix in a follow-up.

What changed: capital first word; full stops; **its** → **it’s** isn’t even needed once you say it plainly; “latter” → **later** (latter = the second of two things, later = afterward); say *what* you fixed instead of “stuff” and “some other thing” so the log is searchable. A commit subject is best in the imperative (“Fix...”, “Add...”), short, and specific.

27. **Confusing PR description — cleaned up:**

This PR changes the order service so it doesn’t crash when the address is missing. I also refactored and removed some dead code. There are still a few known bugs, but this is a good checkpoint.
Please review.

What changed: “change” → **changes** (the PR is singular); “dont” → **doesn’t**; add **the** before “address”; **There are still a few bugs** (countable → “a few”, not “much”); split the run-on; **Please** and “review” spelled out (no “plz”); capital **I**.

28. **Your original line — corrected:**

Can you do research on grammar for me and teach me? I already know it, but right now I'm making so many mistakes.

What changed: “u → **you**”; “reaseach → **research**”; “grammer → **grammar**”; it’s a request, so it ends with **?**; capital **I**; “i already know” needs an object → “I already know **it**”; “as of now” → **right now**; **so many mistakes** (countable plural) instead of “so much mistake.” That last one is the single most common mistake in your writing — when you can count it, use **many** and add the **-s**.

You’ve now touched every rule in the course in one sitting. The three habits that will buy you the most respect in commits, PRs, and standups: **(1)** end every sentence with a full stop, **(2)** capital **I** always, and **(3)** count it before you write **much** vs **many**. Keep this file bookmarked and redo it in a week — if you score the same drills clean twice, the rules have stuck.

Part of the Developer English course — see [00-index.md](#).